

에이전트는 대화로 일하지 않는다



SKILL.state 정독 — 이력을 버리고 상태를 남기는 실행 설계

- A. 병이 뭔가 — 대화 이력의 이차 팽창과 문맥 오염
- B. 약이 뭔가 — 세 개의 입력, 상태 전이, 추론의 즉시 폐기
- C. 증거 — 통제 벤치마크와 실전 벤치마크, 예산 매칭 대조
- D. 계보 — 도구 · ReAct · MemGPT · 압축 · 대화 상태 추적 · 시험대

차례

들어가며 · 실행은 더 이상 추론 문제가 아니다	4
01 제1장 · 대화가 곧 기억이라는 설계	7
02 제2장 · 세 개의 입력	10
03 제3장 · 상태는 어떻게 자라나나	13
04 제4장 · 복잡도의 산술	17
05 제5장 · 재현 가능한 세계	20
06 제6장 · 평평한 프롬프트	23
07 제7장 · 시끄러운 방과 무음의 붕괴	28
08 제8장 · 실전에서도 통하는가	32
09 제9장 · 짧아서 좋은 게 아니다	35
10 제10장 · 도구를 쥐는 손	39
11 제11장 · 이력의 계보	42
12 제12장 · 긴 문맥의 착각과 압축의 함정	45
13 제13장 · 대화 상태 추적에서 온 유전자	49
14 제14장 · 시험대의 설계	52
15 제15장 · 작은 모델은 어디서 무너지나	55
16 제16장 · 이 설계가 무너지는 세 지점	58

DAY

들어가며 · 실행은 더 이상 추론 문제가 아니다

“원문: SKILL.state: Scalable Long-Horizon Agent Skills — arXiv:2608.26263 · Sanket Badhe, Priyanka Tiwari (Google LLC), Jonghyun Chung (Google LLC · Purdue University)”

“링크: <https://arxiv.org/abs/2608.26263>”

I 이 책이 읽는 논문

에이전트에 대한 이야기는 대개 두 부류로 갈립니다. 더 똑똑하게 만드는 이야기와 더 많이 연결하는 이야기입니다. 모델을 바꾸거나, 도구를 붙이거나, 여러 에이전트를 조립하거나. 이 논문은 다른 자리를 겨냥합니다. 이미 똑똑한 모델이 이미 주어진 절차를 **길게** 실행할 때, 그 실행을 담는 그릇이 무엇인지를 묻는 것입니다.

지금의 에이전트 런타임은 거의 예외 없이 대화(conversation)를 그릇으로 씁니다. 매 순간 모델은 원래 지시문과 함께, 지금까지의 생각·행동·관찰·도구 출력이 몽땅 쌓인 기록을 다시 읽습니다. 한 발짝 실행할 때마다 프롬프트가 길어지고, 지난 관찰과 죽은 추론이 문맥 한구석에 남아 모델을 혼듭니다. 논문의 진단은 정확합니다. 실행이 길어지는 순간 실행은 추론의 문제가 아니라 **시스템의 문제**가 된다는 것입니다.

SKILL.state의 답은 단호합니다. 대화 기록을 아예 그릇으로 쓰지 않는 것. 매 단계 모델이 받는 입력은 세 가지로 축소됩니다 — 바뀌지 않는 절차 명세, 현재의 구조화된 실행 상태, 그리고 최신 관찰 하나. 모델이 내놓은 추론은 상태 갱신이 검증되는 순간 버려집니다. 다음 단계는 지나온 이야기를 재생하지 않고, 현재 상태에서 다시 시작합니다.

결과는 두 축에서 나타납니다. 정확도는 오르거나 유지되고, 토큰은 폭풍처럼 줄어듭니다. 100단계 실행에서 누적 토큰이 16.2배 차이가 벌어지고, 200단계에서는 정확도 0.94를 유지하면서 경쟁 기법은 수백만 토큰을 태우는 동안 12만 토큰로 끝납니다. 관찰이 오염되는 소음 아래에서도, 세계가 몰래 바뀌는 붕괴 아래에서도 무너지지 않습니다.

I 왜 굳이 한 편으로 책을 만드는가

숫자만 놓고 보면 이 논문은 또 하나의 효율화 기법처럼 읽힙니다. 그러나 이 논문이 손대는 지점 — 실행 상태를 어디에 두는가 — 는 에이전트 설계 전반을 관통하는 축입니다. 이 축을 제대로 보려면 이 논문이 등장한 풍경을 함께 읽어야 합니다. 그래서 이 책은 본론 한 편을 정독하면서, 그 논문이 대화하고 있는 관련 연구를 통째로 부록이 아니라 본문으로 읽습니다.

- 도구를 쥐는 계보는 어디까지 왔고, 정작 실행은 왜 빈칸으로 남았는가
- ReAct는 왜 이력을 쌓는 설계가 되었고 무엇을 얻었는가
- MemGPT와 Mem0는 기억을 어떻게 다루려 했고 어디까지 갔는가
- Lost in the Middle은 긴 문맥이 실제로 어떻게 부서지는지를 무엇으로 보였는가
- LLMingua 같은 압축은 왜 이 논문의 실험에서 참혹하게 무너지는가
- 대화 상태 추적(DST)이라는 오래된 학문은 이 설계의 어떤 유전자인가
- 결과를 낳은 두 시험대는 각각 어떤 실패를 유혹하도록 설계되었는가

이 일곱 갈래 읽기가 10장부터 14장까지 이어지고, 본 논문의 설계·실험은 1장부터 9장까지, 소형 모델의 실패와 한계·실무는 15장부터 17장까지 놓았습니다. 논문 한 편이 세계 하나를 어떻게 여는지 보여주는 구성입니다.

I 이 책의 읽는 법

각 장의 머리에 논문의 원문 출처를 달았습니다. 본 논문 장들은 논문의 절 구성을 따라가되, 표와 그림은 책용으로 다시 그렸습니다. 그림의 모든 수치는 논문 표의 실측치를 그대로 옮긴 것이고, 제 해석이 들어가는 대목은 장마다 구분해서 밝힙니다. 수식은 논문의 표기를 그대로 씁니다.

10장부터 14장의 관련 논문 읽기는 본 논문의 참고문헌이 가리키는 연구들을 제가 정리한 것입니다. 본 논문의 숫자가 아닌 부분에서는 제 요약과 제 읽기가 섞이므로, 원 논문을 직접 확인할 때 이 책의 인용을 딛돌 삼지 말고 디딤돌로 삼으시길 바랍니다.

표기에 대한 약속 하나. 이 책은 SKILL.state를 비롯한 런타임 이름을 그대로 씁니다. 프롬프트(ReAct)는 이력을 계속 붙이는 표준형, 메모리(요약)는 요약으로 이력을 압축하는 형, 스테이트풀(LangGraph형)은 구조 상태를 문맥에 함께 주입하는 형을 가리키는 논문의 베이스라인 이름입니다. 1장에서 다시 정의합니다.

이 책이 독자에게 바라는 것은 하나입니다. 에이전트가 느려지고, 비싸지고, 이상해지는 이유를 모델 탓으로만 돌리던 습관을 한 번 접는 것. 그리고 “지금 상태가 뭔가”를 묻는 습관을 얻는 것. 그 습관이 이 책 전체의 결론이기도 합니다.

DAY

01

제1장 · 대화가 곧 기억이라는 설계

“원문: SKILL.state — arXiv:2608.26263 · §1 Introduction”

한 줄 요약

에이전트 런타임은 지나온 이야기를 통째로 다시 읽으며 일하는데, 이 관습이 긴 실행에서 비용과 오류를 동시에 키운다 — 논문의 진단은 여기서 시작한다.

에이전트는 어떻게 일하는가

언어 모델은 대화 상자 안에서 살았습니다. 사람이 말을 걸면 답하고, 대화가 길어지면 위에 쌓인 문맥을 다시 읽으며 다음 말을 잇습니다. 에이전트는 이 대화 상자를 일터로 개조한 것입니다. 도구를 호출하는 것도, 파일을 읽는 것도, 터미널에 명령을 치는 것도 모두 대화의 한 턴으로 치환됩니다. 관찰은 사용자 발화처럼 입력되고, 행동은 모델의 응답처럼 출력됩니다. ReAct 이후 이 구조는 사실상 표준이 되었습니다.

짧은 일이라면 아무 문제가 없습니다. 대화 기반 실행은 구현이 쉽고, 디버깅은 기록을 위에서 아래로 읽으면 되고, 모든 것이 한 문서에 있으니 설명력도 좋습니다. 에이전트 프레임워크들이 이 구조를 따르는 데는 이유가 있습니다.

그러나 일이 길어지면 이 그릇의 성질이 바뀝니다. 대화는 더하는 것밖에 못하는 구조입니다. 한 번 입력된 관찰은 지워지지 않고, 한 번 적힌 추론은 문맥에 남습니다. 10턴 짜리 일에서는 장점이던 완전한 기록이, 200턴 짜리 일에서는 매 턴 전부를 다시 읽어야 하는 짐이 됩니다. 논문의 표현을 빌리면 — 프롬프트 크기가 실행 길이와 함께 자라나고, 쓸모를 잃은 관찰과 죽은 추론이 문맥 한구석에 계속 남아 모델로 하여금 **현재의 사실과 역사의 유물을 끊임없이 가려내게 만듭니다**.

병의 이름 — 이차 팽창과 문맥 오염

논문은 이 관습의 문제를 두 가지로 정리합니다.

첫째는 **비용의 이차 팽창**입니다. 매 턴의 입력이 그때까지의 전체 기록이라면, 턴이 늘어날수록 한 턴의 입력은 커지고, 그 커지는 입력을 턴 수만큼 다시 더하니 전체 비용은 실행 길이의 제곱으로 자랍니다. 3장과 4장에서 수식으로 다루겠지만, 표지 없이 감만 짚어도 — 100턴의 일에서 1턴째의 관찰은 100번 읽힙니다. 한 번 보면 되는 것을 백 번 사는 셈입니다.

둘째는 **문맥 오염**입니다. 오래된 관찰은 사실이 아니라 옛날 사실입니다. 창고에서 물건이 옮겨졌다면 세 턴 전의 “1번 선반에 있다”는 기록은 이제 거짓말입니다. 그런데 대화 기반 런타임은 이 거짓말을 지우지 않습니다. 모델은 매 턴 최신 관찰과 옛 기록 사이에서 어느 쪽을 믿을지 스스로 재판해야 하고, 이 재판이 길어지는 실행에서 틀릴 확률을 키웁니다. 논문의 실험 3은 이 재판이 실제로 어떻게 환각으로 이어지는지를 보여줍니다 — 세계가 조용히 바뀌면 이력 기반 런타임은 다섯에서 여덟 턴 동안 옛 기록을 믿고 엉뚱한 행동을 합니다.

이 두 문제는 메모리 시스템으로 완화할 수 있습니다. 관찰을 요약해 두거나, 필요할 때 꺼내 보는 검색창을 달거나. 그러나 논문이 짚는 요점은 날카롭습니다. 이런 완화책도 결국 같은 실행 의미론을 공유한다는 것입니다. 미래의 결정이 **지난 실행의 텍스트 재구성**에 조건지어져 있다는 뜻에서, 근본이 아니라 증상을 다루는 것이라는 것입니다.

실행이 시스템 문제가 되는 순간

논문 서두의 문장 하나가 이 책의 출발점입니다. “에이전트가 길게 도는 절차를 점점 더 실행하게 되면서, 실행 자체가 순수한 추론 문제가 아니라 시스템 문제가 되었다.”

소프트웨어 공학은 이 문제를 압니다. 프로그램이 커지면 코드를 통째로 읽는 해석기로는 못 버티고, 상태를 레지스터와 메모리에 유지하고 명령 포인터만 옮기는 기계가 필요해졌습니다. 대화형 실행은 매 순간 프로그램 전체를 처음부터 다시 해석하는 해석기와 같습니다. 실행 상태를 명시적으로 들고 다니는 기계가 필요한 시점 — 그것이 이 논문이 선을 긋는 자리입니다.

한 걸음마다 전부를 다시 읽는다 — 이력의 누적

대화형 런타임: 매 턴의 관찰·추론·행동이 입력에 남는다 · 턴 t의 프롬프트

관찰 추론 행동

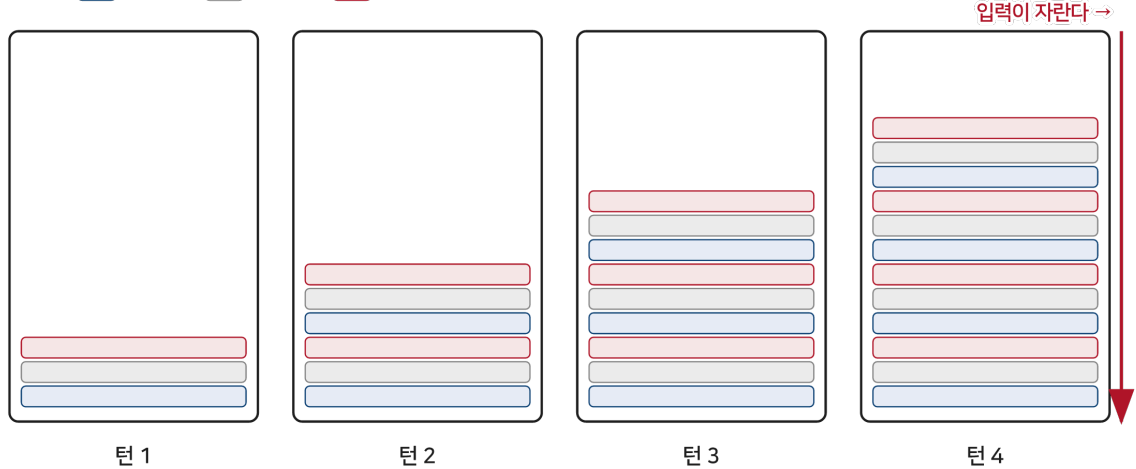


그림 1-1 이력의 누적 — 한 걸음마다 입력이 자란다

논문이 세운 과제

이 진단 위에서 논문은 과제를 뒤집습니다. 이력을 어떻게 잘 압축할까가 아니라, **이력 없이 실행할 수 있는가**입니다. 실행에 정말 필요한 정보만 남기고 나머지를 버리면, 입력은 길이와 무관해지고, 오염도 사라집니다. 남는 질문은 그릇의 모양입니다 — 무엇이 실행에 필요한 정보인가.

논문의 답이 SKILL.state입니다. 그릇을 대화에서 상태로 바꾸는 것. 2장에서 이 설계의 핵심인 세 개의 입력을, 3장에서 상태가 어떻게 자라나고 검증되는지를 봅니다. 그리고 그 설계가 진짜인지 시험하는 다섯 개의 실험이 6장부터 9장까지 이어집니다.

이 장의 돌멩이 하나

논문을 읽기 전에 짚어둘 것이 있습니다. 이 논문은 기억 시스템을 부정하지 않습니다. 요약도, 검색도, 장기 기억도 각자 쓸모가 있습니다. 이 논문이 부정하는 것은 그것들을 **실행의 기본 의미론**으로 삼는 관습입니다. 에이전트가 일하는 동안의 단기 기억은 텍스트 더미가 아니라 구조라야 한다는 것 — 이 구분이 이후 모든 장을 관통합니다. 11장에서 이 관습을 만든 ReAct를, 12장에서 그 관습을 완화하려던 압축과 긴 문맥 연구를 다시 만나게 됩니다.

DAY

02

제2장 · 세 개의 입력

“원문: SKILL.state — arXiv:2608.26263 · §3 SKILL.state (구조 개요)”

한 줄 요약

매 실행 단계에서 모델은 불변 명세와 현재 상태와 최신 관찰만 받는다 — 이 세 가지가 다음 행동을 결정하는 데 필요한 전 부라는 설계 선언이다.

하나의 식

SKILL.state의 실행은 한 줄의 식으로 요약됩니다. 실행 단계 t 에서 모델이 받는 입력 A_t 는 다음과 같습니다.

$$A_t = (P, \Sigma_t, O_t)$$

P 는 절차 명세(procedural specification), Σ_t 는 단계 t 의 구조화된 실행 상태, O_t 는 최신 관찰입니다. 눈여겨볼 것은 식에 없는 것입니다. 지난 관찰도, 지난 행동도, 지난 추론도 없습니다. 모델은 지나온 이야기를 전혀 받지 않습니다.

이 세 가지가 하는 일은 각각 다릅니다. P 는 처음부터 끝까지 바뀌지 않는 규칙서입니다 — 누가, 무엇을, 어떤 행동 공간으로 실행하는지. Σ_t 는 지금까지의 실행이 남긴 결과물의 목록입니다 — 발견한 사실, 해본 시도, 열어본 파일. O_t 는 방금 환경에서 도착한 하나의 소식입니다. 규칙서와 장부와 오늘의 편지. 일하는 데 필요한 것이 이 셋뿐이라는 주장입니다.

추론은 연료가 아니라 연기

이 설계에서 가장 급진적인 결정은 추론의 취급입니다. SKILL.state는 모델의 사고 자체를 무시하지 않습니다. 오히려 단계 안에서의 다단계 사고 — 여러 걸음의 논리를 차곡차곡 쌓는 연쇄 추론 — 를 온전히 허용합니다. 모델은 관찰을 해석하고, 상태를 읽고, 다음 행동을 고르는 데 필요한 만큼 충분히 생각합니다.

그러나 그 사고는 산출물을 낸 뒤 버려집니다. 모델이 내놓는 것은 세 묶음입니다 — 추론 흔적 R_t , 상태 갱신 $\Delta\Sigma_t$, 그리고 실행할 행동 a_t .

$$(R_t, \Delta\Sigma_t, a_t)$$

상태 갱신이 결정론적 검증을 통과해 적용되는 순간 R_t 는 영구히 폐기되고, 이후 어떤 프롬프트에도 다시 나타나지 않습니다. 논문의 표현을 옮기면, 추론은 상태 전이를 만들어내는 **중간 계산**일 뿐입니다. 잠깐 타오르고 남는 것은 재가 아니라 상태라는 것 — 이 은유가 이 설계 전체를 요약합니다.

이 취급이 반직관적으로 들린다면, 인간의 일을 떠올려 보십시오. 창고 관리자는 어제 무슨 생각을 하며 물건을 옮겼는지 기억하지 못해도 일합니다. 선반의 현재 배치와 오늘 도착한 출하시지만 있으면 됩니다. 그가 기억해야 하는 것은 추론이 아니라 추론의 결과입니다. SKILL.state는 에이전트에게 같은 직무 규정을 부과한 셈입니다.

실행은 상태 전이다

이 그릇 교체가 실행의 본질에 대한 재정의의 동반합니다. 대화형 런타임에서 실행은 **이야기의 누적**입니다. SKILL.state에서 실행은 **상태의 전이**입니다. 매 단계는 현재 상태에서 다음 상태로 옮겨 가는 한 걸음이고, 이야기는 그 걸음의 부산물일 뿐입니다.

이 재정의가 주는 귀결은 강력합니다. 실행의 정확성이 이야기를 얼마나 잘 재생하는지에 달리지 않고, 상태가 얼마나 참인지에 달립니다. 다음 장에서 이 상태가 어떤 스키마로 짜이고, 어떻게 검증되고, 어떻게 병합되는지를 봅니다. 4장에서는 이 구조가 비용을 어떻게 선형으로 떨어뜨리는지 계산합니다.

SKILL.state — 세 개의 입력, 버려지는 추론

입력은 명세·상태·최신 관찰뿐 · 논문 그림 1 재구성

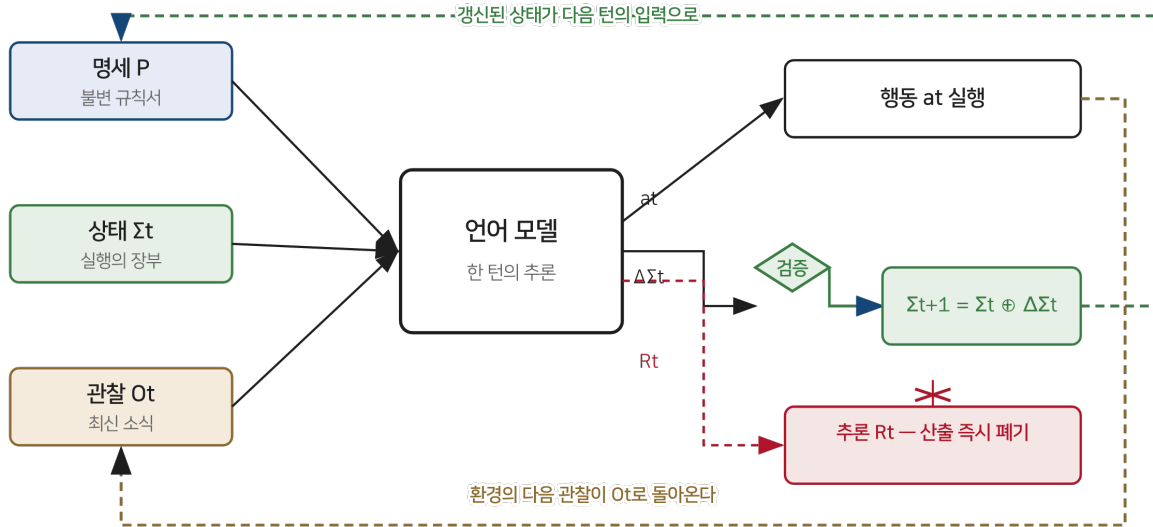


그림 2-1 SKILL.state 아키텍처 — 세 개의 입력과 버려지는 추론

베이스라인의 세 얼굴

논문은 이 설계를 세 개의 기존 패러다임과 나란히 놓고 시험합니다. 이름이 헛갈릴 수 있어 여기서 정의합니다.

프롬프트(ReAct형) — 표준 관습. 모든 관찰·추론·행동을 계속 붙이는 전사본입니다.

메모리(요약형) — 과거를 자연어 요약으로 압축해 최근 세 턴과 함께 다는 방식. MemGPT 계열의 아이디어를 벤치마크 용으로 정리한 것입니다.

스테이트풀(LangGraph형) — 구조 상태 블록을 문맥에 주입하되, 대화 전사본을 그대로 유지합니다. 상태를 보조 자료로 더하는 방식입니다.

뉘앙스가 보이십니까. 셋 다 이력이 기본 그릇입니다. 메모리는 이력을 요약하고, 스테이트풀은 이력에 상태를 엮습니다. SKILL.state만 이력을 버립니다. 베이스라인의 이 배열이 논문의 실험 해석을 함께 결정합니다 — 상태를 더하는 것과 이력을 버리는 것은 다른 일입니다. 9장의 예산 매칭 실험이 이 차이를 정면으로 시험합니다.

나의 해석 — 충분통계량으로서의 상태

이 장의 읽기 하나를 남깁니다. $A_t = (P, \Sigma_t, O_t)$ 는 통계학의 오래된 개념을 에이전트에 이식한 것으로 읽힙니다. 미래에 필요한 모든 것이 지금의 요약 안에 들어 있다는 **충분통계량**의 생각. 논문은 이 말을 쓰지 않지만 16장에서 다룰 한계 서술 — “과거가 미래에 영향을 주는 모든 것이 알려지는 즉시 상태로 투영될 수 있다면 폐기는 무손실이다” — 는 정확히 충분통계량의 정의 문장입니다. 이 읽기는 13장의 대화 상태 추적 계보와 이어지고, 16장의 세 가지 붕괴 지점도 모두 충분통계량 가정이 무너지는 지점들입니다. 책의 뼈대가 되는 읽기이므로 여기서 심어둡니다.

DAY

03

제3장 · 상태는 어떻게 자라나나

“원문: SKILL.state — arXiv:2608.26263 · §3.1~3.2, 알고리즘 1, 부록 A.4·B.3”

한 줄 요약

상태는 모델이 제안하고 런타임이 검증한다 — 이 역할 분담이 잘못된 상태 갱신이 영구 장부를 오염시키는 일을 원천적으로 막는다.

저자는 도메인당 한 번 쓴다

2장의 세 입력 중 실제로 자라나는 것은 Σ_t 하나입니다. 이 상태가 어떤 모양이어야 하는지가 설계의 첫 문제입니다. 논문의 대답은 “일마다 쓰지 말고 도메인마다 한 번 쓰라”는 것입니다.

상태 스키마는 도메인의 성질에서 나옵니다. 창고를 다룬다면 재고의 위치가, 저장소를 다룬다면 브랜치와 풀 리퀘스트의 상태가, 고객 응대를 다룬다면 주문과 환불의 내역이 미래의 실행을 좌우하는 정보입니다. 논문이 드는 예가 인상적입니다 — InterCode CTF 벤치마크의 서로 다른 100개 도전 과제 전체에서 에이전트는 **정적 5필드 스키마 하나**를 재사용합니다. 발견한 플래그(discovered_flags), 시험해 본 가설(tested_hypotheses), 열어본 파일(active_files), 작업 디렉터리(working_dir), 명령 요약(cmd_summary). 과제마다 상태를 새로 설계하는 것이 아니라, 도메인의 골격을 한 번 잡으면 그대로 쓴다는 것입니다.

이 결정의 무게는 17장의 실무 장에서 다시 다룹니다. 여기서는 방향만 짚습니다 — 상태 설계는 코딩이 아니라 도메인 모델링이며, 한 번 잘 하면 여러 번 수확이 나온다.

한 턴의 해부

실행의 한 바퀴를 순서대로 뜯어봅니다. 논문의 알고리즘 1을 글로 옮기면 다음과 같습니다.

먼저 런타임이 최신 관찰 O_t 를 받습니다. 이어서 세 입력으로 프롬프트를 구성해 모델을 부릅니다. 모델은 추론하고, 세 묶음 $(R_t, \Delta\Sigma_t, a_t)$ 을 냅니다. 런타임은 상태 갱신 $\Delta\Sigma_t$ 를 **결정론적으로 검증**합니다. 통과하면 상태에 병합하고, 행동 a_t 를 실행합니다. 그리고 다음 관찰을 받아 같은 일을 반복합니다.

여기서 구체적인 냄새가 나는 부분 두 곳을 논문 부록에서 가져옵니다. 먼저 실제 실행 프롬프트의 골격입니다 — SKILL.state 런타임이 매 턴 모델에게 주는 지시는 대략 이런 모양입니다.

```
Instructions:
{skill.instructions}

Skill Execution State:
{현재 상태의 JSON}

Latest Observation: {최신 관찰}

Provide your response with:

1. Step-by-step reasoning (will be discarded after execution)

2. A JSON block containing both your State Patch and your Action.
The JSON block MUST have exactly these two keys:
{ "state_patch": { <dict: your state updates, set keys to null to delete> },
  "action": "<string: the exact command you want to execute>" }
```

프롬프트 자체가 설계를 웅변합니다. 상태는 통째로 직렬화되어 들어가고, 사고는 “실행 뒤 버려질 것”이라고 미리 공지되고, 산출 계약은 정확히 두 개의 키로 못 박힙니다. 다음은 창고 환경에서의 실제 한 턴입니다. “Customer ordered item_12”라는 관찰이 왔을 때 모델의 산출은 이랬습니다.

```
{
  "state_patch": {
    "inventory": {
      "shelf_42": null
    }
  },
  "action": "Ship item_12 shelf_42"
}
```

모델의 사고 — “item_12는 shelf_42에 있으니 출하하고 선반을 비워야 한다” — 는 이 턴이 끝나면 사라집니다. 남은 것은 선반 42번이 비었다는 사실 하나입니다. 추론의 그림자가 아니라 추론의 결론만이 다음 턴으로 건너갑니다.

병합 연산자와 삭제의 문법

상태 갱신은 덮어쓰기가 아니라 병합입니다. 논문은 갱신을 다음처럼 씁니다.

$$\Sigma_{t+1} = \Sigma_t \oplus \Delta\Sigma_t$$

$\oplus$ 는 null-삭제 의미론을 가진 딕셔너리 병합 연산자입니다. 갱신 조각의 키 값이 null이면 그 키를 상태에서 지우고, 그 외에는 값을 덮어쓰거나 더합니다. 위의 예에서 “shelf_42”: null이 정확히 이 문법입니다 — 출하 완료라는 사실을 “선반 42번 삭제”라는 연산으로 표현한 것입니다.

이 작은 문법 선택이 우연이 아닌 이유가 15장에서 드러납니다. 소형 모델의 1위 오류가 기존 키를 갱신에 누락시켜 통째로 날려버리는 것이었기 때문입니다. 갱신을 “새 상태 전체를 쓰기”가 아니라 “바뀐 부분만 말하기”로 계약하면, 실수의 폭발 반경이 턴 하나로 좁아듭니다.

검증은 런타임의 몫

설계에서 가장 조용하지만 가장 중요한 분업이 여기 있습니다. 상태 갱신을 **제안**하는 것은 모델이지만, 스키마 소유와 **검증**은 결정론적 런타임에 있습니다. 모델이 흥터 난 JSON을 내놓으면 그 갱신은 적용되지 않습니다. 영구 상태가 오염되는 길이 원천적으로 닫혀 있는 것입니다. 잘못된 제안은 롤백-재시도 주기를 돌 뿐입니다.

이 분업의 대비가 선명합니다. 대화형 런타임에서는 모델의 실수가 곧 문맥의 실수입니다 — 잘못된 결론이 이력에 적히면 그것이 사실로 재생됩니다. SKILL.state에서 실수는 상태에 도달하기 전에 걸러집니다. 형식 오류는 시스템이, 의미 오류는 다음 관찰이 잡습니다. 7장의 상태 회복 실험에서 이 구조가 무음 붕괴 앞에서 어떻게 행동하는지 봅니다.

한 턴은 검증을 통과해야 상태에 남는다

논문 알고리즘 1의 실행 주기 — 통과·실패·폐기의 세 갈래

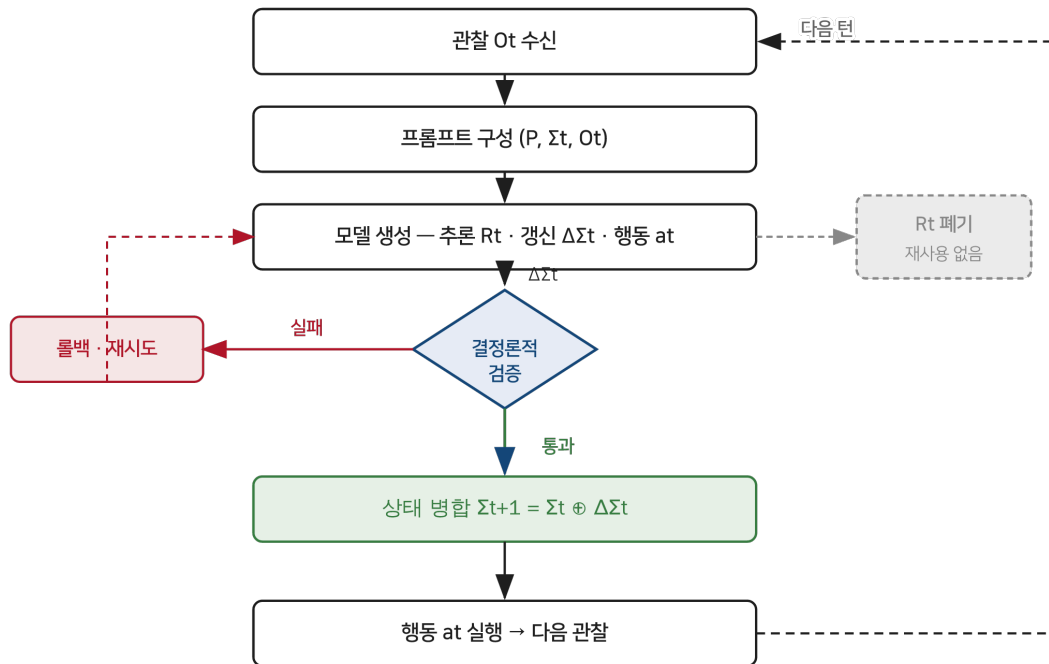


그림 3-1 한 턴의 실행 주기 — 검증을 통과해야 상태에 남는다

나의 해석 — 두 겹의 기억 시스템

이 장의 구조를 한 걸음 물러나 보면, SKILL.state는 기억을 두 겹으로 나눈 시스템입니다. 딱딱한 겹겹질 — 검증과 병합을 책임지는 결정론적 런타임 — 과 무른 속 — 매 턴 새로 태어나는 추론. 컴퓨터 구조의 레지스터와 연산 회로가 떠오르는 배치입니다. 무른 쪽이 틀려도 딱딱한 쪽이 온전하면 시스템은 다음 턴에 재시도할 기회를 갖습니다.

MemGPT가 이 문제를 접근한 방식과 겹쳐 보면 대비가 재미있습니다. MemGPT는 모델 스스로가 자기 기억을 편집하는 쪽으로 설계를 밀었습니다 — 지능이 높을수록 기억 관리도 잘하리라는 베팅입니다. SKILL.state는 반대로 베팅합니다 — 기억의 무결성은 지능에 맡기지 않는다. 11장에서 이 두 베팅의 갈림길을 다시 만납니다.

DAY

04

제4장 · 복잡도의 산술

“원문: SKILL.state — arXiv:2608.26263 · §3.3 Complexity Analysis”

한 줄 요약

대화형 실행의 누적 비용은 지평의 제곱으로 자라고 SKILL.state는 선형이다 — 이 대목의 계산은 간단하지만 그 함의는 무겁다.

제공이 나오는 이유

복잡도 이야기는 자취수 하나에서 출발합니다. 대화형 런타임에서 단계 t 의 프롬프트 길이 $|C_t|$ 는 그때까지 쌓인 상호작용 전부이므로 $\mathcal{O}(t)$ 입니다. 한 턴의 입력이 그 턴 번호에 비례해서 자란다는 뜻입니다.

전체 비용은 이것을 실행 지평 T 까지 더한 것입니다.

$$\sum_{t=1}^T |C_t| = \mathcal{O}(T^2)$$

1부터 T 까지의 합이 $T^2/2$ 로 자란다는 고등학교 산수입니다. 다만 이 산수의 주어가 돈과 시간이라는 것이 문제입니다. 100턴의 일에서 첫 관찰은 백 번 결제되고, 마지막 관찰은 한 번 결제됩니다. 같은 정보에 붙는 가격이 순번에 따라 백 배까지 벌어지는 구조입니다.

상수가 이기는 자리

SKILL.state의 계산은 대구를 이룹니다. 매 단계의 입력은 명세와 상태와 관찰뿐이므로 그 길이는

$$|P_t| = \mathcal{O}(|P| + |\Sigma| + |O|)$$

입니다. 이 값은 지금까지 몇 턴을 돌았는지 t 와 무관합니다. 실행이 아무리 길어져도 상태가 담는 정보의 양이 자라지 않으면 프롬프트도 자라지 않습니다. 논문이 이를 $\mathcal{O}(1)$ 프롬프트 풋프린트라고 부릅니다. 누적 비용은

$$\sum_{t=1}^T |P_t| = \mathcal{O}(T)$$

입니다. 턴 수에 정비례. 비즈니스 언어로 번역하면 — 대화형 실행은 실행 길이가 두 배가 될 때 비용이 네 배가 되는 구조고, SKILL.state는 두 배가 되면 비용도 두 배가 되는 구조입니다.

숫자로 만져보는 격차

추상적 기호보다 논문의 실측이 생생합니다. 참고 환경에서 SKILL.state의 평균 프롬프트는 지평 10에서 200까지 어떤 값으로 흘러가는지 — 1,775, 1,736, 1,773, 1,905, 그리고 1,811 토큰. 지평이 스무 배 늘어나는 동안 입력은 1,736에서 1,905 사이의 좁은 밴드를 오갑니다. 상태 스키마에 새 사실이 더해지는 속도와 낡은 사실이 null-삭제되는 속도가 거의 맞아떨어지기 때문입니다.

반대편 베이스라인의 같은 줄기를 봅니다. 지평 100에서 스테이트풀 베이스라인의 누적은 1,062,387 토큰, SKILL.state는 65,408 토큰 — **16.2배** 차이입니다. 지평 200에 이르면 메모리 베이스라인이 6.18백만 토큰을 태우는 동안 SKILL.state는 122,384 토큰으로 끝납니다. 제공과 선형의 차이는 짧은 지평에서는 소수점 차이로 숨어 있다가, 지평이 길어질수록 자릿수로 드러납니다. 이 실측 곡선은 6장의 그림에서 로그 축 위에 그대로 펼쳐집니다.

제공은 면적이고 선형은 길이다

같은 10턴의 실행 — 이력은 10×10 격자를, 상태는 한 줄을 청구한다

이력 기반 — $O(T^2)$

상태 기반 — $O(T)$

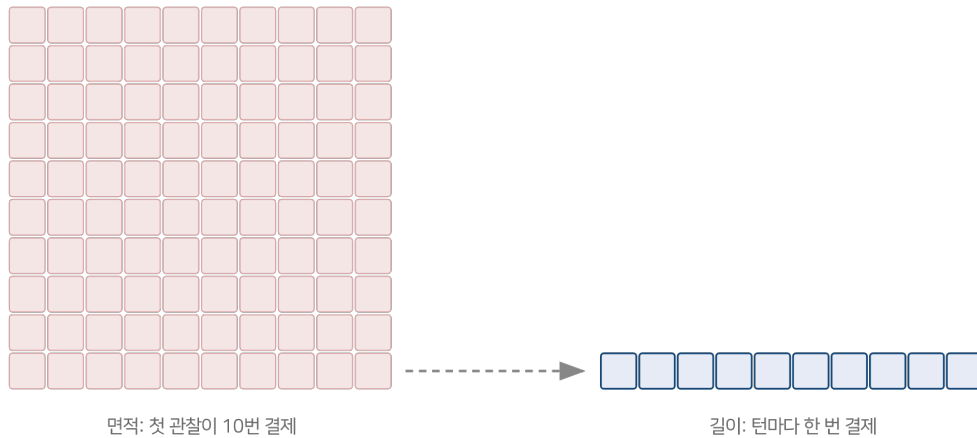


그림 4-1 제공은 면적이고 선형은 길이다

나의 해석 — 복잡도는 정직한 언어다

이 장의 계산을 대수롭지 않게 여길 수도 있습니다. 어차피 요즘 모델은 긴 문맥을 받고, 캐시도 있고, 가격도 내린다고. 그러나 이 반론은 두 겹에서 무너집니다.

첫째, 창이 넓다고 해서 팽창이 공짜가 되지 않습니다. 긴 문맥의 검색 능력은 위치와 길이에 따라 부식됩니다 — 12장의 Lost in the Middle이 보이는 바로 그 현상입니다. 제공 팽창은 비용 문제이기 전에 정확도 문제로 침투합니다. 둘째, 프롬프트 캐싱은 반복 접두사에 대한 값의 문제이지 팽창 자체의 해부가 아닙니다. 6장에서 보겠지만 지평 200에서 이력 기반 런타임의 정확도 자체가 무너집니다 — 캐시는 그 무너짐을 할인해 주지 않습니다.

그래서 이 장의 산술을 나는 이렇게 읽습니다. 복잡도는 설계의 정직성을 묻는 언어다. 어떤 설계가 시간이 지나며 무엇에 지수를 붙이는지를 보면, 그 설계가 진짜 믿는 바가 보인다. 이 논문이 지수를 붙이는 곳은 턴 수 단 하나이고, 지수는 1입니다. 그 단순함이 설계의 신앙 고백처럼 읽히는 대목입니다.

이 장에서 가져갈 것

시스템을 설계할 때 “이 구성요소는 무엇의 함수로 자라나는가”를 묻는 습관. 대화 기록이라면 답은 턴 수입니다. 로그라면 답은 사건 수입니다. 캐시라면 답은 고유키 수입니다. 각각의 성장 지수를 적어 보는 것만으로, 오늘의 데모가 내일의 장애로 바뀌는 지점을 미리 볼 수 있습니다. 17장의 실무 장에서 이 질문이 상태 설계 체크리스트의 첫 항이 됩니다.

DAY

05

제5장 · 재현 가능한 세계

“원문: SKILL.state — arXiv:2608.26263 · §4.1, 부록 B”

한 줄 요약

SkillExecBench는 운에 좌우되지 않는 결정론적 세계에서 실행 역학만을 격리해 시험한다 — 두 환경은 서로 다른 병의 근육을 요구한다.

왜 새 벤치마크인가

런타임의 우열을 재는 일은 뜻밖으로 어렵습니다. 실전 과제는 열려 있어서 같은 문제를 두 번 만날 수 없고, 성공 여부가 탐색의 운에 좌우되면 무엇이 실행 역학 덕이고 무엇이 운인지 갈라지지 않습니다. 논문이 자체 벤치마크 SkillExecBench를 만든 이유입니다. 목적은 문제 풀기 실력을 재는 것이 아니라 **실행의 관리 능력**을 격리하는 것 — 문제 자체는 단순하되, 단 순해야 상태 관리의 실패가 정확하게 곧바로 번지기 때문입니다.

설계의 축은 세 가지입니다. 순차적 — 사건이 정해진 순서로 하나씩 도착합니다. 상태 유지 — 한 사건의 처리가 이전 사건들의 결과에 의존합니다. 결정론적 — 같은 시드면 같은 세계가 재생됩니다. 다섯 개의 절차 생성 시드로 모든 런타임이 정확히 같은 사건 열을 겪게 하는 것으로 공정함을 확보합니다.

두 개의 세계

벤치마크는 성질이 대비되는 두 환경을 담습니다.

환경 1 · 창고 관리. 500개의 독립된 선반에 물건이 드나드는 이산 재고 세계입니다. 행동은 네 가지 — 보관(Store), 출하(Ship), 이동(Move), 대기(Wait). 관찰은 텍스트 알림으로 도착합니다 — 출하 입고, 고객 주문, 선반 정비. 이 환경이 시험하는 것은 **독립 변수의 장기 유지**입니다. 500개 선반의 상태는 서로 얽히지 않지만, 초기의 배치 정보가 창 밖으로 밀려난 뒤에도 정확해야 합니다. 빠뜨리면 안 되는 사실이 많고, 그 사실들이 오래가는 세계입니다.

환경 2 · 소프트웨어 저장소. Git 브랜치, 커밋, 풀 리퀘스트, CI 테스트 상태가 얽힌 관계 그래프입니다. 행동은 체리픽(CherryPick), 병합(Merge), 테스트 실행(RunTests), 릴리스 생성(CreateRelease), 롤백(Rollback). 관찰은 CI 웹hook과 코드 리뷰 코멘트입니다. 이 환경이 시험하는 것은 **얽힌 구조 위의 추론**입니다. PR 하나를 병합하면 대상 브랜치와 의존 PR들의 상태가 함께 바뀌는 세계 — 단일 행동의 파급이 그래프 전체로 퍼집니다.

두 환경을 한 문장으로 대조하면 이렇습니다. 창고는 기억의 지구력을 보고, 저장소는 기억의 구조 감각을 봅니다.

구분	창고 관리	소프트웨어 저장소
세계의 모양	500개 독립 선반의 평면	브랜치·커밋·PR·CI의 관계 그래프
행동 공간	Store · Ship · Move · Wait	CherryPick · Merge · RunTests · CreateRelease · Rollback
관찰	출하 입고·고객 주문·정비 알림	CI 웹hook·리뷰 코멘트·이슈 배정
시험 대상	독립 변수의 장기 유지	밀집 의존 관계의 구조 추론
채점	유효 행동 비율 (결정론적 시뮬레이션과 대조)	깨끗이 병합된 요청의 비율

채점 방식도 통제되어 있습니다. 창고에서는 결정론적 시뮬레이션이 만드는 정답 궤적과의 대조로 성공 행동 수를 세고, 저장소에서는 CI를 깨뜨리지 않고 마스터에 병합된 요청의 비율을 셉니다. “얼마나 맞혔는가”가 아니라 “얼마나 유효하게 처리했는가” — 실행기의 채점은 이쪽이 맞습니다.

세계를 만드는 코드

논문 부록의 작업 생성 의사코드는 벤치마크의 철학을 압축합니다. 시드를 고정한 난수 생성기로 사건 유형을 뽑되, 가능한 사건은 세계의 현재 상태에서 도출합니다 — 빈 선반이 없으면 입고는 일어나지 않고, 점유된 선반이 없으면 주문도 정비도 일어나지 않습니다. 사건이 세계를 바꾸고, 바뀐 세계가 다음 사건의 가능성을 좁힙니다. 상태와 사건이 서로를 규정하는 이 구조 자체가 상태 중심 세계관의 축소판입니다.

두 환경 — 평면의 지구력, 그래프의 파급

창고는 독립 변수의 장기 유지를, 저장소는 얽힌 의존의 구조 추론을 시험한다

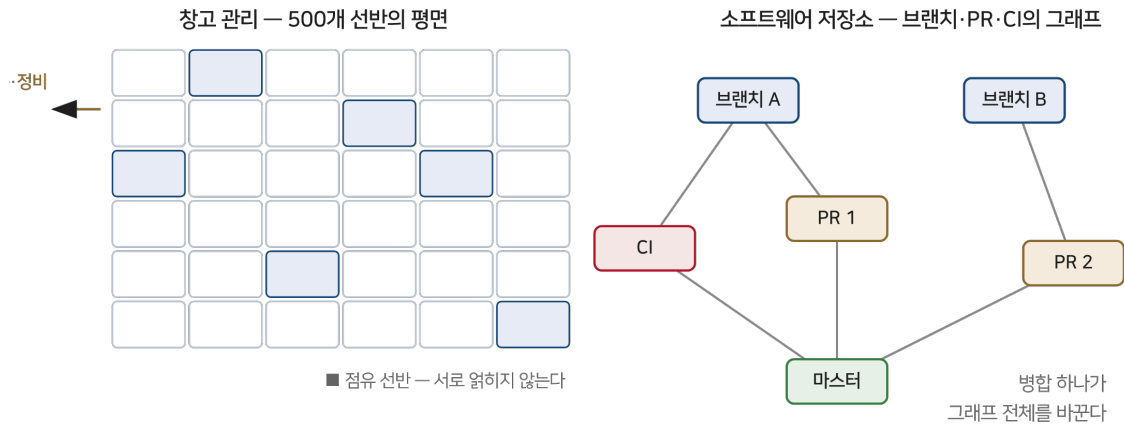


그림 5-1 두 환경 — 선반 격자의 지구력, 관계 그래프의 파급

나의 해석 — 통제와 실전의 이중주

이 장의 벤치마크를 평가하는 잣대는 “얼마나 진짜 같은가”가 아니라 “얼마나 정확히 격리했는가”여야 합니다. SkillExecBench는 의도적으로 단순하고 의도적으로 결정론적입니다. 그 대가로 이 벤치마크가 주는 결론은 인과가 분명합니다 — 같은 세계, 같은 사건, 같은 채점 아래에서 벌어지는 차이는 오직 실행 관리 방식의 차이입니다.

동시에 논문은 이 약점을 알고 있어서 실전 벤치마크 두 개를 곁들입니다 — 8장의 InterCode CTF와 τ -Bench입니다. 통제된 세계에서 얻은 인과를, 통제 없는 세계에서 다시 확인하는 이중주. 실험 논문을 읽을 때 이 배치가 갖춰져 있는지를 보는 습관이 논문 감별의 기본기입니다. 다음 장부터 이 무대 위에서 다섯 개의 실험이 시작됩니다.

DAY

06

제6장 · 평평한 프롬프트

“원문: SKILL.state — arXiv:2608.26263 · §5.1~5.2, 표 1”

한 줄 요약

지평을 10에서 200까지 늘려도 SKILL.state의 입력은 그 자리에 있고 정확도는 오히려 역전한다 — 첫 실험의 풍경이다.

실험의 문법

본격적인 결과 읽기에 앞서 논문의 실험 문법을 정리합니다. 비교 대상은 2장에서 정의한 세 베이스라인 — 프롬프트(ReAct형), 메모리(요약형), 스테이트풀(LangGraph형)입니다. 실행 모델은 세 족속에서 뽑습니다 — 구글의 Gemini-3-Flash, 개방 가중치의 Gemma-4-31B-it와 Qwen-3-8B-it. 디코딩은 온도 0.0, top-p 1.0으로 못 박아 재현성을 확보하고, 통제 실험은 다섯 개의 절차 생성 시드로 돌려 평균과 표준편차를 냅니다. 지평 50 이상에서의 격차는 짝비교 t 검정으로 p가 0.01 미만 — 우연의 여지가 통계적으로 닫혀 있다는 뜻입니다.

첫 실험의 독립변수는 하나 — 실행 지평을 10, 25, 50, 100, 200 단계로 늘리는 것입니다. 종속변수는 셋 — 정확도, 평균 프롬프트 크기, 누적 토큰.

표 전체를 놓고 보면

참고 환경, Gemini-3-Flash의 전체 결과입니다. 논문 표 1을 그대로 옮겼습니다 (평균 ± 표준편차, 5 시드).

지평	런타임	정확도	평균 프롬프트	총 토큰
10	프롬프트(ReAct)	0.90 ± 0.02	3,249 ± 94	9,438 ± 371
10	메모리(요약)	1.00 ± 0.00	3,300 ± 123	9,972 ± 204
10	스테이트풀(LangGraph)	1.00 ± 0.00	3,430 ± 42	10,337 ± 299
10	SKILL.state	1.00 ± 0.00	1,775 ± 74	5,870 ± 131
25	프롬프트(ReAct)	0.92 ± 0.02	6,052 ± 192	42,689 ± 2,238
25	메모리(요약)	0.99 ± 0.00	6,357 ± 203	43,067 ± 1,948
25	스테이트풀(LangGraph)	1.00 ± 0.00	5,858 ± 301	41,238 ± 3,196
25	SKILL.state	1.00 ± 0.00	1,736 ± 49	14,714 ± 564
50	프롬프트(ReAct)	0.88 ± 0.04	11,931 ± 346	171,658 ± 6,978
50	메모리(요약)	0.93 ± 0.03	7,582 ± 283	131,455 ± 6,841
50	스테이트풀(LangGraph)	0.94 ± 0.00	11,594 ± 438	170,992 ± 7,918
50	SKILL.state	0.96 ± 0.01	1,773 ± 53	30,151 ± 1,231
100	프롬프트(ReAct)	0.84 ± 0.07	36,362 ± 1,304	1,245,413 ± 53,241
100	메모리(요약)	0.87 ± 0.05	29,607 ± 978	1,082,154 ± 83,212
100	스테이트풀(LangGraph)	0.91 ± 0.02	31,354 ± 831	1,062,387 ± 53,839
100	SKILL.state	0.94 ± 0.01	1,905 ± 93	65,408 ± 5,431
200	프롬프트(ReAct)	0.74 ± 0.14	48,007 ± 2,092	2,608,755 ± 102,415
200	메모리(요약)	0.84 ± 0.09	84,364 ± 3,446	6,175,509 ± 294,089
200	스테이트풀(LangGraph)	0.88 ± 0.03	72,305 ± 3,096	5,041,164 ± 346,925

지평	런타임	정확도	평균 프롬프트	총 토큰
200	SKILL.state	0.94 ± 0.02	$1,811 \pm 184$	$122,384 \pm 4,522$

표를 세로로 읽으면 4장의 이론이 그대로 실측이 됩니다. 세 베이스라인의 총 토큰은 지평의 제공에 맞물려 자라다 못해 지평 200에서는 요약 방식이 오히려 앞서던 ReAct를 추월해 6.18백만 토큰을 찍습니다 — 요약문이 눈덩이처럼 불어나는 한계상황입니다. SKILL.state의 같은 열은 5,870에서 122,384로, 지평에 거의 정비례합니다.

누적 토큰 소비 — 이차 팽창대 선형 성장 (로그 축)

참고 환경 · Gemini-3-Flash · 논문 표 1 · 가로축은 실행 지평 T (로그)

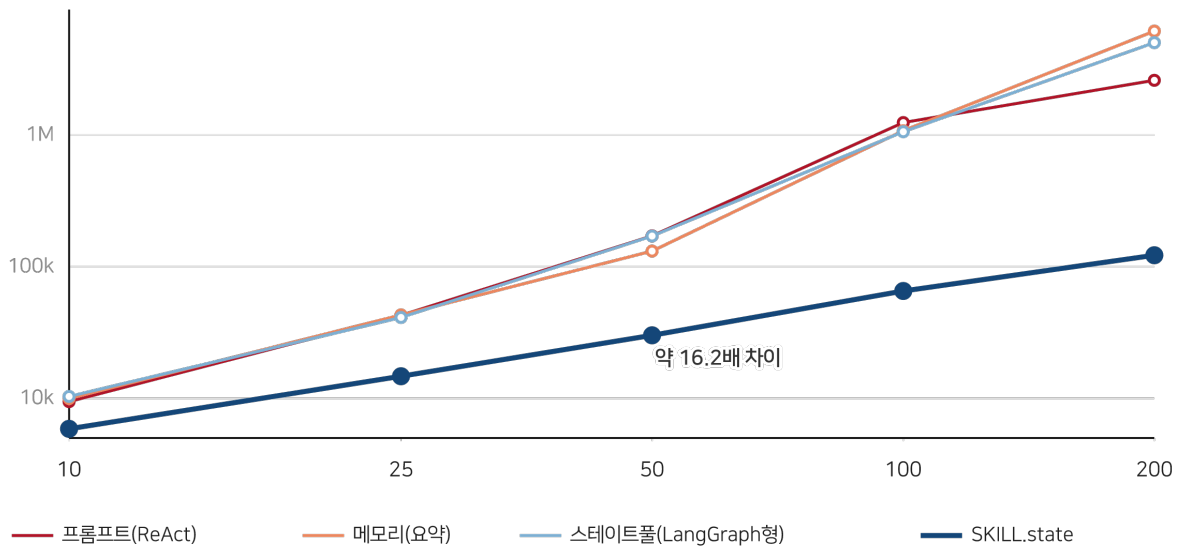


그림 6-1 누적 토큰 소비 — 로그 축 위의 제공 곡선과 선형 직선

로그 축은 자릿수의 언어입니다. 그림 6-1에서 세 베이스라인의 곡선은 우측으로 갈수록 가파라지는 제공의 문양이고, SKILL.state의 직선은 지평과 무관한 기울기로 내려그립니다. 지평 100에서 벌어지는 약 16.2배의 세로 간격이 이 책 전체를 관통하는 숫자입니다.

입력은 그 자리에 있다

평균 프롬프트의 흐름을 따로 보는 이유는, 이것이 텀당 체감 — 지연과 문맥 건강 — 과 직결되기 때문입니다.

평균 프롬프트 크기 — SKILL.state는 평평하다

턴당 입력 토큰 · 논문 표 1 · SKILL.state는 지평과 무관하게 약 1,700~1,900 토큰

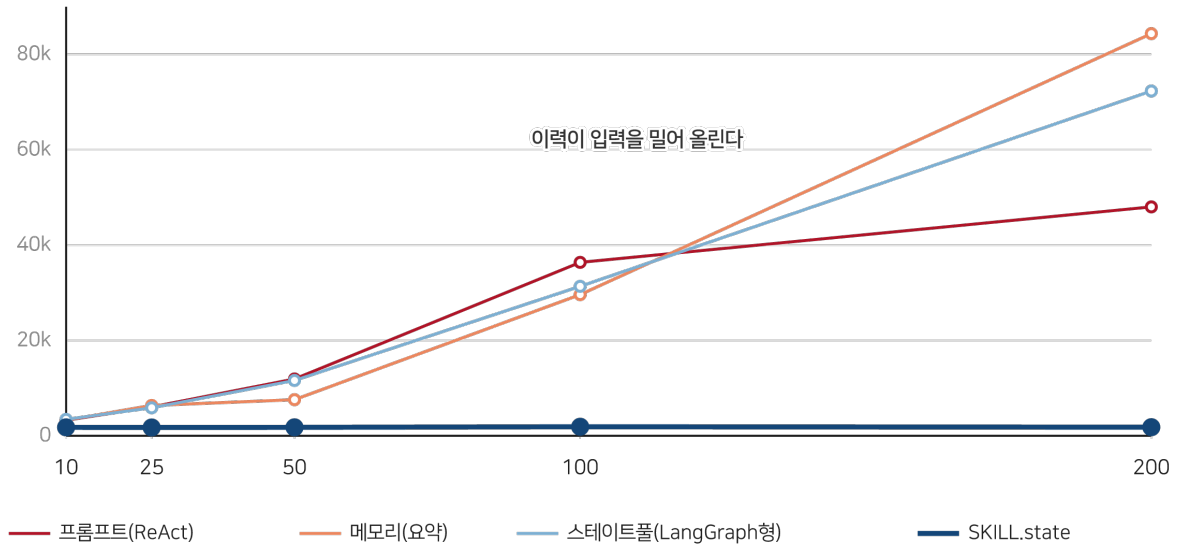


그림 6-2 평균 프롬프트 크기 — SKILL.state는 지평과 무관하게 평평하다

그림 6-2에서 세 베이스라인은 3천 토큰대에서 출발해 3만~8만 토큰대로 올라갑니다. SKILL.state는 어떤 지평에서도 1,736~1,905 토큰의 밴드 안에 있습니다. 이 밴드의 존재가 단순한 절약이 아니라는 것이 다음 장의 주제입니다 — 입력이 자라지 않으면 소음도, 낡은 사실도, 죽은 추론도 쌓일 곳이 없습니다.

길어져도 무너지지 않는다

비용이 아니라 실력의 문 — 정확도입니다.

정확도 — 길어져도 무너지지 않는다

참고 환경 · 논문 표 1 · 이력 기반 런타임은 T가 늘수록 하락

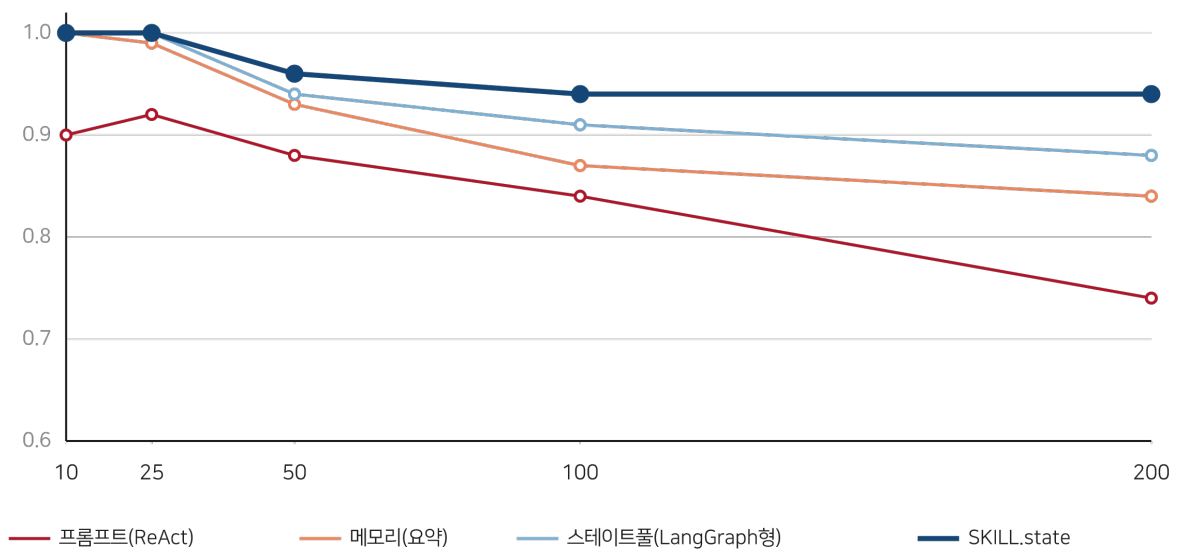


그림 6-3 정확도의 궤적 — 지평이 늘수록 벌어지는 역전

그림 6-3의 문양을 언어로 옮기면 이렇습니다. 지평 10에서는 넷 다 0.90~1.00 — 짧은 일에서는 관습도 충분합니다. 격차는 50부터 벌어지고 100에서는 역전이 자릿수로 굳습니다. 지평 200에서 ReAct는 0.74로 내려앉고 편차도 ± 0.14 로 흔들리는 반면, SKILL.state는 0.94를 유지합니다 — 지평 100과 같은 값입니다. 실행이 길어지는 것이 SKILL.state에게는 추가 비용조차 아니라는 뜻에서, 이 수치는 “유지”가 아니라 “면역”에 가깝습니다.

부록의 저장소 환경(표 6)도 빼대는 같습니다 — 지평 100에서 SKILL.state가 0.78, 베이스라인은 0.53~0.63. 참고보다 전체적으로 낮은 점수는 그래프 추론의 난이도를 반영하고, 그 어려운 세계에서도 격차의 방향은 그대로입니다.

나의 해석 — 표준편차도 데이터다

이 장의 표에서 제 눈이 먼저 가는 곳은 평균이 아니라 편차 열입니다. 지평 200의 ReAct는 ± 0.14 — 시드 다섯 개에서 0.6대와 0.8대가 섞였다는 뜻입니다. 무엇이 같았을까요. 제 추측은 이렇습니다 — 이력이 길어지면 어떤 사건 열에서는 재생이 성공하고 어떤 열에서는 과거의 유물이 이깁니다. 재생의 성패가 사건 순서의 주사위에 달렸다는 것이 바로 결정론이 아니라 우연에 의존하는 실행의 정의입니다. 반면 SKILL.state의 편차는 지평 200에서 ± 0.02 입니다. 상태만 남는 구조에서는 사건 순서가 흔들어도 결론이 흔들리지 않습니다. 평균은 어느 쪽이 나은지를 말하고, 편차는 어느 쪽이 믿을 만한지를 말합니다 — 운용에서는 후자가 먼저입니다.

이 장에서 가져갈 것

당신의 에이전트가 긴 일을 맡는다면 오늘 밤 재볼 지표 두 개 — 턴당 입력 크기의 추이와, 실행 길이를 늘렸을 때 성공률의 기울기. 입력이 턴 수에 비례해 자라는 그래프라면 4장의 제공 청구서가 이미 발행되고 있는 것입니다. 두 지표는 로그 한 줄이면 담을 수 있고, 이 그림들의 문양과 나란히 놓기만 해도 진단이 됩니다.

DAY

07

제7장 · 시끄러운 방과 무음의 붕괴

“원문: SKILL.state — arXiv:2608.26263 · §5.3~5.4, 표 2·3·9·10, 부록 C”

한 줄 요약

오염된 관찰 앞에서는 걸러내는 힘이, 세계가 소리 없이 바뀌는 앞에서는 잇는 힘이 시험받는다 — SKILL.state는 둘 다 구조로 해결한다.

시끄러운 방 — 실험 2

실전의 관찰은 깨끗하지 않습니다. 시스템 텔레메트리, 무관한 로그, 지나가는 이벤트가 진짜 소식과 섞여 도착합니다. 논문은 지평을 50으로 고정하고 턴당 노이즈 사건을 5개, 20개, 50개씩 끼워 넣습니다. 노이즈의 제조법이 치밀합니다 — 값은 매 단계 무작위로 다시 뽑고, 의미는 과제와 철저히 무관하며, 세계의 실제 상태를 절대 바꾸지 않는, 관찰 전용 방해 물입니다. 참고에는 로봇 배터리 잔량·온도 센서·카메라 인식 로그가, 저장소에는 접속조차 안 된 서버의 시스로그가 “— BACKGROUND TELEMETRY —” 꼬리표를 달고 꼬리처럼 따라붙습니다.

노이즈	런타임	점수
5개 (낮음)	프롬프트(ReAct)	0.68
	메모리(요약)	1.00
	스테이트풀(LangGraph)	1.00
	SKILL.state	1.00
20개 (중간)	프롬프트(ReAct)	0.61
	메모리(요약)	1.00
	스테이트풀(LangGraph)	0.98
	SKILL.state	0.97
50개 (높음)	프롬프트(ReAct)	0.53
	메모리(요약)	0.96
	스테이트풀(LangGraph)	0.98
	SKILL.state	0.98

참고(표 2)에서 ReAct형이 0.68에서 0.53으로 미끄러지는 동안 SKILL.state는 0.97 이상을 지킵니다. 논문의 설명은 기계적입니다 — 방해물이 상태 갱신 생성 단계에서 걸러지고, 걸러진 이상 다음 프롬프트에 영원히 들어가지 못합니다. 소음에 대한 면적이 필터의 성능이 아니라 구조의 성질이라는 것.

저장소 환경(표 9)은 드라마가 더 큼니다. 노이즈 0에서 50으로 가는 동안 ReAct형이 0.76에서 0.11로 몰락하고, 요약형도 0.85에서 0.74로 내려갑니다. SKILL.state는 0.90에서 0.80으로 — 내리긴 하지만 무너지지는 않습니다. 구조가 어려운 세계일수록 소음이 묻히는 곳도 깊어진다는 것, 그리고 상태의 독은 그 안에서도 높이를 유지한다는 것.

노이즈 강건성 — 오염된 관찰을 흘려보내지 않는다

T=50 고정 · 노이즈는 턴당 무관계 사건 수 · 논문 표 2(창고)·표 9(저장소)

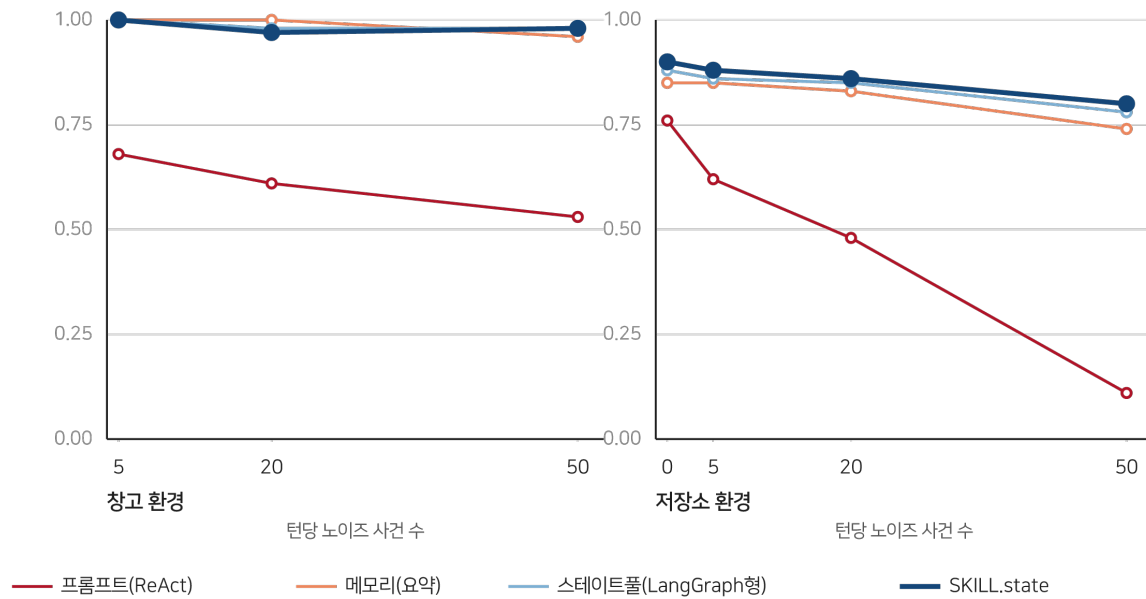


그림 7-1 노이즈 강건성 — 두 환경에서 무너지는 이력, 유지되는 상태

무음의 붕괴 — 실험 3

더 무서운 시나리오는 소음이 아니라 침묵 속의 변경입니다. 에이전트의 행동 루프 밖에서 — 외부 행위자가 — 세계가 바뀌는 경우입니다. 창고의 물건이 몰래 옮겨지고, 저장소의 브랜치가 강제 푸시되는 상황. 논문은 보정 알림이 도착한 뒤 에이전트가 몇 턴을 헤매는지로 회복력을 잽니다.

시나리오	런타임	성공	회복 단계
A · 은밀 감사	세 베이스라인	예	5~8
	SKILL.state	예	0
B · 은밀 바코드	세 베이스라인	예	6~8
	SKILL.state	예	0
C · 은밀 이동	세 베이스라인	예	5~8
	SKILL.state	예	0
D · 취소된 주문	전부	아니오	해당 없음

결과가 노골적입니다. 이력 기반 런타임은 5~8턴 동안 환각합니다 — 프롬프트에 남은 옛 사실이 모순되는 새 관찰을 이기기 때문입니다. 저장소 환경(표 10)에서는 이 지연이 더 길어져 강제 푸시 앞에서 8~12턴, 불안정 CI 앞에서 9~14턴을 헤맵니다. SKILL.state의 회복 단계는 전 시나리오에서 0입니다. 결정이 현재 상태에만 의존하므로 보정 알림이 상태를 갱신하는 즉시 다음 행동이 올바른 행동이 됩니다. 잊는 것이 곧 적응인 구조입니다.

세계가 바뀐 뒤 — 환각의 길이가 회복력이다

보정 알림 도착(턴 0) 이후 · 논문 표 3·10



그림 7-2 회복 타임라인 — 환각의 길이가 회복력이다

나의 해석 — 두 실험은 같은 이야기의 앞뒤다

이 장의 두 실험을 따로 보면 각각 잡음 제거와 결함 허용의 교훈으로 읽히지만, 나는 한 문장으로 읽습니다 — **과거가 미래를 이기는가, 현재가 미래를 이기는가**. 노이즈 실험에서 지는 쪽은 무관한 과거에게 이기는 쪽은 현재의 관찰입니다. 회복 실험에서 지는 쪽은 뒤집힌 과거에게 이기는 쪽은 뒤집힌 현재입니다. 두 실험 다, 이력이 곧 권위가 되는 구조의 취약성을 반대 방향에서 두 번 찌른 것입니다.

운영의 언어로 바꾸면 더 실감이 납니다. 에이전트가 사람의 개입을 받아들이는 일 — 계획 변경, 외부 수정, 긴급 취소 — 은 전부 무음의 붕괴입니다. “취소된 주문” 시나리오에서 전 런타임이 실패한 것도 시사적입니다. 회복이 빠른 시스템은 사람과 일할 수 있고, 회복이 느린 시스템은 사람의 수정을 자기 서사와의 충돌로 겪습니다. 협업의 유연성이 기억의 유연성에서 나온다는 것 — 이 장의 교훈을 조직에 옮기면 이렇게 됩니다.

이 장에서 가져갈 것

실험 재현은 어렵지 않습니다. 당신의 에이전트에 오늘 중 한 번, 실행 중간에 세계를 바꿔 보십시오 — 파일을 직접 수정하거나, 데이터를 지우거나, 계획을 철회하거나. 몇 턴 만에 적응하는지 세어 보면 됩니다. 다섯 턴을 넘게 헤맨다면 그 에이전트의 결정은 상태가 아니라 이야기를 보고 있다는 뜻입니다. 그리고 그 진단이 내려진 순간, 고칠 곳도 정해집니다 — 17장의 절차를 따라 바뀐 사실이 이야기가 아니라 상태로 들어오는 길을 트십시오.

DAY

08

제8장 · 실전에서도 통하는가

“원문: SKILL.state — arXiv:2608.26263 · §4.2-5.5, 표 4”

한 줄 요약

통제된 세계의 결론은 통제 없는 세계에서도 성립한다 — 리눅스 침입 시험과 기업 고객 응대에서 SKILL.state는 정확도와 비용을 함께 가져간다.

두 개의 실전 벤치마크

5장의 SkillExecBench가 병리 실험실이라면 이 장의 두 벤치마크는 야전입니다. 과제가 무엇인지 모르는 채 탐색해야 하고, 세계가 결정론적이지 않으며, 성공이 점수가 아니라 실제 결과로 판명나는 세계입니다.

InterCode CTF. 도커 컨테이너 안의 리눅스 터미널에서 100개의 캡처 더 플래그 도전을 푸는 벤치마크입니다. 역공학, 포렌식, 암호 해독, 바이너리 익스플로잇이 섞여 있고, 에이전트는 bash 명령을 치며 가설을 시험하고 숨겨진 플래그를 찾아냅니다. 성공은 이진 판정 — 플래그의 정확한 일치(pass@1)입니다. 이 벤치마크의 본질은 반복 시행의 세계입니다. 같은 명령을 다시 치면 안 되고, 실패한 가설은 기억해 다시 시도하지 말아야 하며, 발견한 단서는 잊지 말아야 합니다. 실행 상태 관리가 점수로 직결되는 구조입니다.

Sierra τ -Bench. 소매(retail)와 항공(airline) 두 도메인의 기업 고객 응대 벤치마크입니다. 에이전트는 시뮬레이션된 고객과 대화하며 관계형 데이터베이스를 도구로 조회하고, 항공권 재예약이나 환불 같은 트랜잭션을 비즈니스 정책의 제약 아래 수행합니다. 채점은 공식 프로그램 채점기가 합니다 — 최종 데이터베이스 상태가 고객 의도를 충족하면서 정책 위반이 없는지를 기계적으로 검증합니다. 데이터베이스 응답이 크고, 사용자가 말을 바꾸고, 정책이 빽빽한 — 문맥이 무겁고 지저분한 세계입니다.

결과 — 세 경기 전판

런타임	CTF pass@1	CTF 토큰	리테일 패스	리테일 토큰	항공 패스	항공 토큰
프롬프트 (ReAct)	43.2%	977k	48.2%	4.48M	21.8%	4.85M
메모리(요약)	46.4%	1.03M	29.9%	4.24M	23.6%	4.65M
스테이트풀 (LangGraph)	41.8%	1.13M	51.7%	3.92M	28.1%	5.28M
SKILL.state	54.2%	387k	58.3%	3.47M	32.4%	2.88M

실전 벤치마크 패스율 — 세 경기 전판 승리

Gemini-3-Flash · 논문 표 4 · InterCode CTF는 pass@1, τ -Bench는 공식 채점기 기준

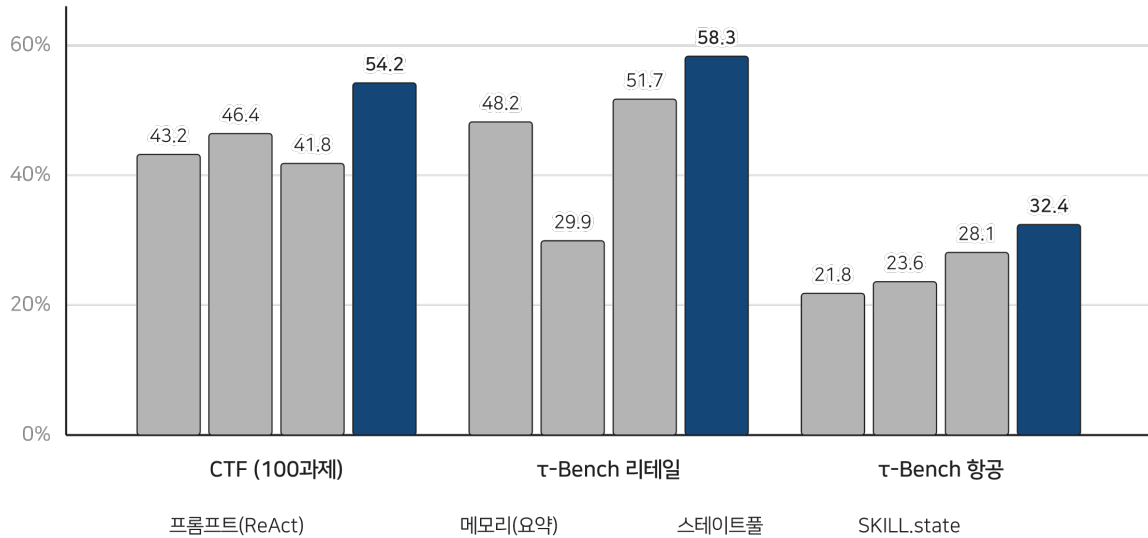


그림 8-1 공개 벤치마크 패스율 — 세 경기 모두 최상위

CTF에서 SKILL.state는 pass@1 54.2%로 최강 베이스라인보다 7.8점, 스테이트풀보다 12.4점 앞섭니다. 동시에 총 토큰은 ReAct 대비 60.4% 줄어든 387k, 스테이트풀 대비로는 65.9% 줄어듭니다. 논문의 해석이 정확합니다 — 발견한 플래그와 시험한 가설을 Σ_t 에 명시적으로 유지하는 것이 실패한 명령의 반복을 막는다는 것. 3장의 5필드 스키마가 점수로 나타난 장면입니다.

τ -Bench 리테일에서는 58.3%로 1위를 달리면서 총 토큰도 3.47M으로 최저입니다. 항공이 가장 드라마틱합니다. 이 도메인은 데이터베이스 응답이 커서 베이스라인의 프롬프트가 한 턴에 11,000 토큰을 넘기로 치솟는데, SKILL.state는 턴당 약 2,800 토큰의 평평한 발걸음을 유지하며 32.4%를 냅니다. 절약 폭은 ReAct 대비 40.5%, 스테이트풀 대비 45.4%.

나의 해석 — 토큰 절감은 부산물이고 반복 억제가 본체

이 장의 숫자를 “비용 절감 기법의 승리”로 읽으면 절반만 읽은 것입니다. 정확도가 오른 이유를 따져야 합니다. CTF의 오류 프로파일을 생각해 보면 — 이력 기반 에이전트의 전형적인 실패는 같은 명령을 다시 치고, 같은 깨진 가설을 다시 굴리고, 이미 열어본 파일을 다시 뒤지는 것입니다. 이걸 기억이 없어서가 아니라 기록이 넘쳐서입니다. 5,000 토큰 깊이의 실패 기록 속에서 “내가 이걸 해봤는가”를 매번 재판하는 것이 실수를 낳습니다.

상태가 작고 명시적이면 이 재판이 필요 없어집니다. tested_hypotheses에 이미 있는 가설은 다시 굴리지 않습니다. 그러므로 이 장의 진짜 메커니즘은 **반복 억제**이고, 토큰 절감은 그 부산물입니다. 순서가 중요합니다 — 절감이 목표였다면 9장의 압축 기법들이 있었고, 그들은 절감에는 성공하고 점수에서 무너졌습니다. 방향이 반대라는 것.

항공 도메인의 교훈도 같은 맥락에서 읽힙니다. 턴당 11,000 토큰의 데이터베이스 응답은 대화형 런타임에게는 흡수해야 하는 홍수지만, SKILL.state에게는 한 번 훑고 필요한 것만 상태로 뽑아내면 끝나는 원료입니다. 큰 관찰 자체는 문제가 아니고, 큰 관찰이 쌓이는 구조가 문제라는 것 — 이 논문의 세계관을 가장 잘 보여주는 사례입니다.

이 장에서 가져갈 것

당신의 에이전트 로그에서 “반복”을 세어 보십시오. 같은 파일을 다시 읽은 횟수, 같은 질의를 다시 날린 횟수, 같은 실패를 다시 한 횟수. 이 숫자가 0에 가깝다면 이력 관리가 잘 되고 있는 것이고, 크다면 상태 부재의 증상입니다. 반복은 눈에 잘 띄는 지표인 데다 개선의 손잡이도 분명합니다 — “해본 일” 목록을 상태로 들고 있는 것만으로 잘 걷힙니다. 17장의 첫 단계가 바로 이 목록입니다.

DAY

09

제9장 · 짧아서 좋은 게 아니다

“원문: SKILL.state — arXiv:2608.26263 · §5.6, 표 5.11”

한 줄 요약

SKILL.state가 이기는 이유가 “프롬프트가 짧아서”라는 반론을 논문은 정면으로 시험한다 — 같은 예산에 묶은 압축 기법들은 전부 무너졌다.

가장 중요한 대조

여기까지 읽었다면 누구나 품게 되는 반론이 있습니다. “짧은 입력이 좋다는 거잖아? 그럼 그냥 짧게 만들면 되는 거 아니야?” 이 반론은 옳은 질문을 담고 있습니다 — SKILL.state의 승리가 **구조화된 상태** 덕분인지, 그저 **입력이 짧다는 사실** 덕분인지를 가려야 하기 때문입니다. 두 설명이 같은 결과를 예측한다면 이론은 절반만 있는 것입니다.

논문의 시험 설계가 훌륭합니다. 참고, 지평 100, Gemini-3-Flash 조건에서 세 압축 구성을 SKILL.state의 예산 — **턴당 약 1,800 토큰** — 에 딱 맞춰 묶습니다. 예산이 같으니 남는 변수는 하나 — **짧게 만드는 방식**입니다.

- 슬라이딩 윈도우 — 최근 턴만 남기고 자른다
- 요약 상한 — 자연어 요약에 딱딱한 토큰 천장을 건다
- ReAct + LLMingua — 작은 모델의 혼란도로 토큰을 통계적으로 가지치기한다

구성	점수	평균 프롬프트	총 토큰
ReAct 전체 이력 (무제한)	0.84	36,362	1,245,413
슬라이딩 윈도우	0.18	1,800	62,100
요약 상한	0.52	1,840	63,400
ReAct + LLMingua	0.22	1,810	62,350
SKILL.state	0.94	1,905	65,408

같은 예산, 다른 결말 — 예산 매칭 대조 (참고 T=100)

모든 압축 구성을 SKILL.state 예산(약 1,800 토큰)에 맞춰도 점수는 같린다 · 논문 표 5

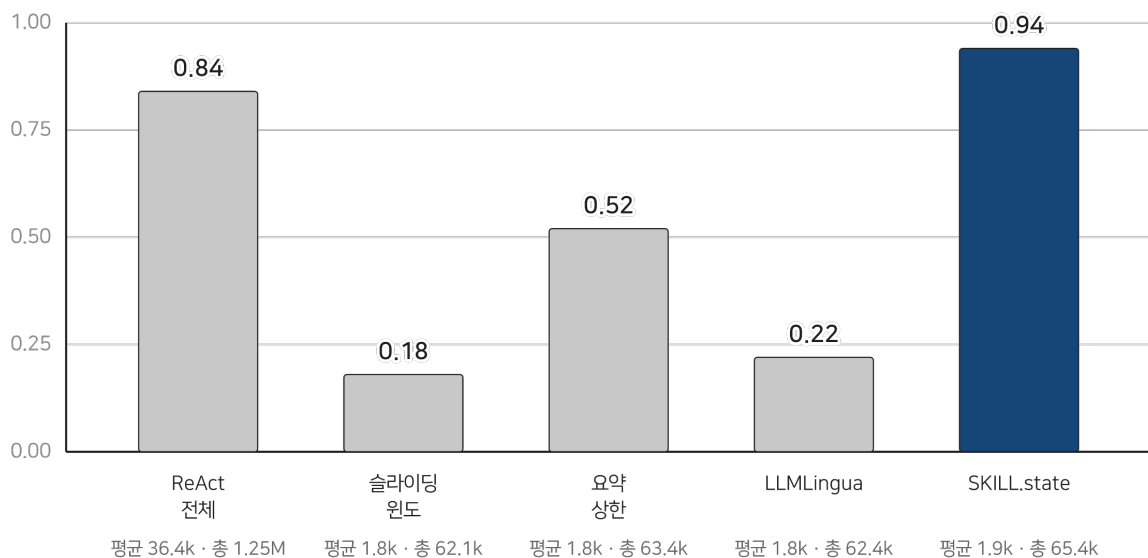


그림 9-1 같은 예산, 다른 결말 — 예산 매칭 대조

무너지는 방식이 기법의 지문이다

결과는 참혹하지만 참혹함에 서명이 있습니다. 각 기법이 왜 무너지는지가 그 기법의 본질을 드러내기 때문입니다.

슬라이딩 윈도우는 0.18로 떨어집니다. 이유는 명확합니다 — 초반의 재고 배치가 창밖으로 쫓겨나기 때문입니다. 창고 과제의 정답은 이른 관찰에 숨어 있고, 최근 턴만 보는 창은 그 관찰이 지나간 뒤에야 문을 닫습니다. **잇는 방향이 정해져 있지 않은 절단은 필연적으로 중요한 것을 자릅니다.**

LLMLingua는 0.22. 통계적 혼란도 기준으로 “복제 가능한” 토큰을 가지치다가 의미상 결정적인 슬롯 식별자들을 지워버립니다. 엔트로피는 중요함의 척도가 아닙니다 — “shelf_42”는 통계적으로 심심한 토큰이지만 창고 세계에서는 그 자체가 정답의 절반입니다. **통계는 구조를 못 지킨다**는 것이 이 0.22의 이름입니다.

요약 상한은 0.52로 그나마 버티지만 여전히 참패입니다. 요약은 사람이 읽기 좋은 산문을 만들지, 실행 가능한 정확한 장부를 만들지는 못하기 때문입니다. “3번 선반쯤에 뭐가 있었던 것 같다”는 요약은 이야기로는 자연스럽고 실행기로는 못쓰입니다.

그리고 같은 예산에서 SKILL.state는 0.94. 6장의 무제한 조건(0.94)과 동일합니다 — 예산에 묶여서도 흔들리지 않았다는 뜻입니다. 짧음이 아니라 **무엇을 남기는지**가 문제였다는 것. 이 대조 하나가 이 논문의 논지를 한 방에 정리합니다.

격차는 지평과 함께 벌어진다

논문 부록의 다중 지평 버전(표 11)이 이 결론을 시간 축으로 확장합니다.

지평	SKILL.state	요약 상한	슬라이딩 윈도우	LLMLingua	ReAct 전체
10	1.00	0.92	0.90	0.88	0.90
25	1.00	0.76	0.62	0.60	0.92
50	0.96	0.64	0.35	0.38	0.88
100	0.94	0.52	0.18	0.22	0.84

지평이 길어질수록 벌어지는 격차 — 예산 매칭 스케일링

참고 · Gemini-3-Flash · 논문 표 11 · 점수

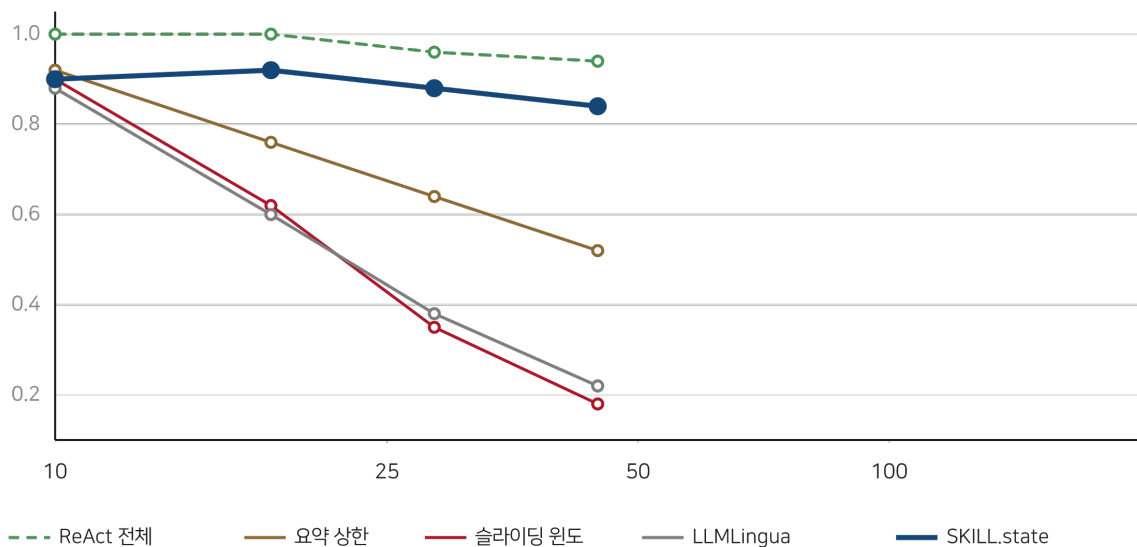


그림 9-2 예산 매칭 스케일링 — 지평이 길어질수록 벌어지는 격차

지평 10에서는 다섯 구성이 0.88~1.00으로 붙어 있습니다. 짧은 일에서는 아무 방식이나 대충 해도 됩니다. 격차는 지평과 함께 벌어져 100에서는 0.18과 0.94로 자릿수가 갈립니다. 절단과 압축의 실패가 우연이 아니라 **법칙**이라는 것 — 기억해야 할 것이 늘어날수록 못 지키는 방식과 지키는 방식의 차이가 커진다는 것.

나와 해석 — 요약과 장부의 존재론

이 장의 실험을 나는 “요약과 장부의 존재론”으로 부릅니다. 요약은 사람을 위한 기억입니다 — 대의를 전하고 뉘앙스를 살리고 흐름을 잇습니다. 장부는 기계를 위한 기억입니다 — 정확하고 갱신 가능하고 조회 가능합니다. 긴 실행을 버티는 것은 장부이고, 요약은 짧은 자리를 메꾸는 재봉틀입니다. 요약 상한이 0.52에서 멈춘 것은 재봉틀로 이불을 짓려 한 값입니다.

실무의 습관을 돌아보게 되는 대목입니다. “지난 대화를 요약해서 넣는다”는 이제 에이전트 제품의 상투구가 됐습니다. 이 장의 수치는 그 상투구의 값을 묻습니다 — 요약이 실행에 필요한 정보를 얼마나 살리는지를 측정해 본 적이 있는가. 요약의 품질을 문장이 자연스러운지로 평가하는 습관, 실행 가능성으로 평가하는 습관을 밀어냈습니다. 12장에서 LLMingua 계열 압축 연구의 전모를, 그리고 이 실험이 그 연구사 위에서 갖는 위치를 다룹니다.

이 장에서 가져갈 것

컨텍스트를 줄이는 방안을 논의할 때 “얼마나 줄었나”와 함께 “무엇을 지켰나”를 물으십시오. 절단은 지키는 것 없이 줄이고, 압축은 통계가 아까워하는 것을 지키고, 상태는 미래가 필요로 하는 것을 지킵니다. 이 세 층위의 구분이 없으면 “토론 90% 절감!”이라는 광고가 0.18의 참사로 도착합니다. 그리고 그 구분을 실측하는 법도 이 장이 보여줍니다 — 같은 예산에 묶어 놓고 점수를 재는 것. 이 예산 매칭 대조는 논문 읽기의 도구로도 남습니다. 어떤 “효율화” 주장도 이 대조 앞에 세워 보면 본심이 드러납니다.

DAY

10

제10장 · 도구를 쥐는 손

“원문 연구: Toolformer(Schick 외, NeurIPS 2023) · ToolLLM(Qin 외, ICLR 2024) · AutoGen(Wu 외, arXiv:2308.08155) · A Systematic Survey of Agent Skills(Badhe-Shah-Tiwari-Kathrotia, 2026) · Agent Skill Security(Badhe-Tiwari, arXiv:2607.13987) — SKILL.state §2.1 참고문헌의 계보”

한 줄 요약

에이전트 기술(skill) 연구는 발견·표현·조합·보안을 다 논했는데 정작 “골라진 기술이 어떻게 실행되는가”는 비어 있었다 — SKILL.state 저자들이 자신들의 서베이에서 파낸 빈칸이다.

저자들은 서베이에서 왔다

이 장을 여는 사실 하나가 이 논문의 배경을 비춘다. SKILL.state의 첫 저자들이 같은 해에 내놓은 것은 이 실행기 논문만이 아니라 — **에이전트 기술에 대한 체계적 서베이**였다. 기술의 생애주기·분류·보안 위협까지 훑은 문헌 종합이다. SKILL.state는 그 서베이 끝에 남은 질문에서 나왔다. 서베이가 그린 지도에는 네 개의 잘 개간된 발이 있었다 — 기술을 **발견**하는 발, **표현**하는 발, **조합**하는 발, **보안**하는 발. 그리고 지도 한복판, 아무도 경작하지 않은 땅 — 골라진 기술이 **어떻게 실행되는**가. 이 논문은 그 빈칸을 메우는 논문이다.

이 맥락을 알면 논문의 조어가 선명해진다. “절차 기술(procedural skill)의 실행 역학”이라는 문제 설정은 하루아침의 발상이 아니라, 저자들이 문헌 전체를 등고선으로 그리며 찾아낸 최저점인 것이다.

2023 · Toolformer — 도구 사용을 가르치는 세 가지 방식

도구를 쥐는 일의 난제는 자명하다. 도구를 쓰면 능력은 늘지만, 언제 쓸지 아는 것 자체가 능력이기 때문이다. Toolformer의 접근이 우아한 이유는 이 문제를 **훈련 문제로 치환**한 것이다. 모델에게 도구 호출 예시를 몇 개 보여주고, 스스로 도구 호출을 삽입한 문장들을 대량으로 생성하게 한 뒤, 그 호출이 이후 토큰의 예측을 실제로 나아지게 하는 경우만 걸러 강화한다. 사람의 주석이 아니라 도구의 유용성 자체가 감별 기준이 된 셈이다. 계산기·백과 검색·달력 같은 소수의 API만으로 이 자기학습을 돌린 결과, 작은 모델이 도구를 쥔 채로 자기보다 훨씬 큰 모델의 맨손질에 필적하는 과제가 생겼다.

이 연구가 계보에 남긴 것은 결과보다 관점이다 — 도구 사용은 프롬프트의 요령이 아니라 **모델에 스며드는 기질**이 될 수 있다는 것. 이후의 도구 사용 연구 대부분이 이 관점의 연장선에 있다.

2024 · ToolLLM — 도구의 세상을 정리하다

ToolLLM은 규모의 응답이다. 개발자들이 실제로 마주하는 도구 세계 — 수만 개의 실제 REST API — 를 그대로 긁어와 지시-계획-응답의 대규모 데이터로 조직하고, 관련 API를 찾아주는 검색기와 결과를 평가하는 채점기를 갖춘 벤치마크까지 묶었다. 도구 사용이 “잘 쓰는가”의 문제에서 “넓은 세계에서 필요한 것을 찾아 쓰는가”의 문제로 옮겨진 것입니다. 표제 부터가 16,000개가 넘는 실제 API를 자랑하는 이 연구는, 도구의 다양성 자체를 학습 과제로 만들었다는 데 의미가 있다.

2023 · AutoGen — 조합의 시대

AutoGen은 같은 질문을 협업 쪽에서 만난다. 대화할 수 있는 단위 — 사람일 수도, 도구일 수도, 모델일 수도 — 를 조립해 프로그램을 짜는 프레임워크. 각 주체의 역할과 능력을 정의하면 나머지는 대화로 굴러가게 하는 설계로, 멀티 에이전트 응용의 사실상 표준 문법이 됐다. 이 계보에서 기술은 쓰이는 단위가 아니라 **주체들 사이의 흐름**이 된다.

여기서 되새길 대목 — 이 조합의 시대에 주체들이 무엇을 주고받는가? 대부분의 프레임워크에서 답은 대화 기록이다. AutoGen이라는 이름 자체가 ‘대화 생성’이다. 조합이 늘수록 오가는 이력도 늘고, 1장의 병은 주체 수의 제곱으로 퍼진다. SKILL.state 논문이 남긴 미완의 숙제 — 다중 에이전트의 공유 상태(16장) — 가 정확히 이 자리다. 조합의 문법은 성숙했는데 조합의 그릇이 아직 대화라는 불일치.

기술 연구의 지형 — 갈린 네 발, 비어 있는 다섯째

저자들의 서베이가 그린 지도에서 SKILL.state는 유일한 백지를 찔다



그림 10-1 기술 연구의 지형 — 갈린 네 발과 비어 있는 다섯째

나의 해석 — 연구의 뒷면을 읽는 법

이 장이 이 책에 들어간 속사정을 밝힌다. 나는 독자에게 논문 읽기의 한 가지 기술을 보여주고 싶었다 — **같은 저자의 이전 논문을 읽으면 논문의 문제가 어디서 왔는지 보인다**. SKILL.state를 단독으로 읽으면 “프롬프트가 비싸니 줄였다”는 정도의 기술적 서사로 읽힌다. 서베이를 곁들이면 다른 이야기가 나온다 — 기술의 생애주기 전체를 등고선으로 그린 사람들이, 지도에서 유일하게 백지인 칸을 발견하고 그 칸을 판 논문이다. 문제의 발견이 해법의 발견보다 어렵고 귀한 일이라면, 이 논문의 절반은 이미 서베이에서 완성되어 있었던 것이다.

보안 서베이를 같이 쓴 저자라는 사실도 군더더기가 아니다. 3장의 설계 — 상태 검증을 결정론적 런타임에 두는 분업 — 는 위협 모델을 골려 본 사람의 손맛으로 읽힌다. 모델의 산출을 신뢰하지만 검증은 시스템이 쥐는 구조는, 프롬프트 주입 이니 이상 데이터니 하는 공격면을 헤아려 본 설계자의 답변처럼 보인다. 다음 장에서는 이 빈칸을 메우는 실행기가 그 앞 세대들 — 이력의 관리자들 — 과 어떻게 다른지를 본다.

DAY

11

제11장 · 이력의 계보

“원문 연구: ReAct(Yao 외, arXiv:2210.03629) · MemGPT(Packer 외, 2023) · MemoryBank(Zhong 외, arXiv:2305.10250) · 생성 에이전트(Park 외, UIST 2023) · Mem0(Chhikara 외, arXiv:2504.19413) · SKILL.state(Badhe 외, 2026, arXiv:2608.26263) — SKILL.state §2.1~2.2 참고문헌의 조상들”

한 줄 요약

SKILL.state가 건너뛴 계단들을 짚어 내려간다 — 이력을 쌓은 ReAct, 이력을 관리한 MemGPT, 기억을 상품화한 Mem0를 읽어야 이 논문의 위치가 보인다.

왜 계보를 읽는가

9장까지 우리는 한 논문의 내부를 걸었다. 이제 걸음을 멈추고 그 논문이 서 있는 땅을 보려 한다. SKILL.state의 참고문헌을 펼치면 대화 상대가 누구인지 보이는데, 그 중에서도 실행과 기억의 설계를 바꿔 온 네 연구 — ReAct, MemGPT, MemoryBank와 Mem0, 생성 에이전트 — 가 이 장의 소재다. 인용의 정확성을 위해 밝혀두면, 이 장의 서술은 각 논문에 대한 내 요약이며 본 논문의 실험 숫자가 아니다.

2022 · ReAct — 루프의 발명

이야기는 ReAct부터다. 요약하면, 언어 모델이 생각(Thought)과 행동(Action)을 번갈아 놓고 행동의 결과(Observation)를 다음 생각의 재료로 삼게 한 것이다. 이 세 층의 교대 — 사고하고, 움직이고, 보고 — 가 오늘날 거의 모든 에이전트 런타임의 문법이 됐다. SKILL.state가 비교하는 “프롬프트(ReAct형)” 베이스라인이 정확히 이 문법이다.

덜 알려진 사실 하나가 이 장의 복선이다. ReAct의 저자들 스스로가 이 문법의 약점을 데이터로 보였다 — 지식 검색 과제에서는 행동을 섞지 않은 순수 사고가 앞서고, 환경과 맞닿은 과제에서야 사고-행동 루프가 빛났다는 것. 루프가 이기는 자리에서 필연적으로 이력이 쌓이기 시작하고, 그 순간 1장의 병 — 프롬프트 팽창과 문맥 오염 — 이 루프의 문법에 새겨진다. ReAct의 위대함과 SKILL.state의 필요는 같은 동전의 양면이다. 이력을 쌓는 문법을 발명했기에, 이력을 버리는 문법이 필요해졌다.

2023 · MemGPT — 창을 운영체제로 보다

이력이 자라는 병이 보이자 다음 세대의 질문은 “어떻게 관리할까”였다. MemGPT의 답변이 아름다운 이유는 은유의 출처다 — 운영체제의 가상 메모리. 주기억장치가 좁으면 디스크로 스왑하듯, 문맥 창이 좁으면 창 밖으로 페이지 아웃하고 필요할 때 페이지 인한다. 모델에게 함수 호출이라는 손을 주어 스스로 자기 기억을 읽고, 쓰고, 편집하게 했다. 요약 주기와 최근 턴의 조합 — SKILL.state 논문의 “메모리(요약형)” 베이스라인의 뿌리가 이곳에 있다.

여기서 계보가 갈라선다. MemGPT의 베팅은 **모델이 기억 관리도 잘하리**라는 것 — 지능이 높아지면 스스로 페이지를 잘 넘기리라는 믿음. SKILL.state의 베팅은 정반대다 — 기억의 무결성은 지능에 맡기지 않고 결정론적 런타임이 천다(3장). 15장에서 보겠지만 소형 모델의 오류 프로파일은 후자의 베팅에 힘을 실어 준다. 다만 MemGPT가 남긴 통찰은 살아 있다 — 문맥 창은 하드웨어가 아니라 **관리 대상 자원**이라는 눈. 그 눈 없이는 SKILL.state도 나오지 않았다.

2023~2025 · MemoryBank와 Mem0 — 기억의 상품화

MemoryBank는 기억에 시간의 화학작용을 들여왔다. 잊곡선 — 반복되지 않은 기억은 강도가 식고, 다시 건드리면 강해진다 — 를 에이전트 기억에 이식해, 뭐든 다 담은 창고가 아니라 휘발과 강화가 있는 은행을 만들었다. 생성 에이전트 연구는 회상의 삼중 척도를 보여줬다 — 최신성, 중요성, 관련성을 곱해 기억 스트림에서 무업을 꺼낼지 고른다는 것. 기억이 저장의 문제가 아니라 **회상 정책의 문제**라는 관점이 이때 자리 잡았다.

Mem0는 이 계열의 운영화다. 기억을 별도 계층으로 떼어내고, 대화가 끝날 때마다 추출 단계와 갱신 단계를 돌려 새 사실을 더할지(ADD), 고칠지(UPDATE), 지울지(DELETE), 돌지(NOOP)를 정한다. 관계형 변형으로 엔터티와 관계의 그래프를 함께 굴리는 등, 기억을 프로젝트로 다루는 설계가 잘 정리되어 있다.

이 계열 전체를 SKILL.state의 눈으로 보면 공통점이 드러난다 — 모두 **장기 기억**을 다룬다. 세션을 넘어, 사용자를 넘어 기억이 가는 곳. 그리고 대부분 실행 중 단기 문맥은 여전히 대화형으로 둔다. SKILL.state는 정확히 그 빈 자리 — 실행 중의 기억 — 를 겨냥한다. 장기 기억이 참고라면 실행 상태는 작업대다. 참고 혁신이 아무리 나아도 작업대가 어지러우면 요리는 실패한다.

LINEAGE

실행 기억의 계보는 네 걸음을 걸었다

쌍고(ReAct), 관리하고(MemGPT), 정책화하고(Mem0), 버린다(SKILL.state).



편집 요약: 이 장의 계보 서술을 재배열한 도식이다.

나의 해석 — 진화가 아니라 분업이다

이 계보를 “부족한 설계가 진보한 계보”로 읽으면 안 된다고 본다. 내 읽기는 **분업의 역사**다. ReAct가 루프를 발명하고, MemGPT가 문맥을 자원으로 격상시키고, Mem0가 장기 기억을 계층으로 분리했다. SKILL.state는 그 분업의 마지막 칸을 채운다 — 세션 사이를 잇는 참고가 아니라 세션 안에서 도는 작업대. 네 연구는 경쟁하는 대답이 아니라 서로 다른 질문에 대한 대답이다.

그래서 실무의 그림도 이렇게 그려진다 — 장기 기억 계층(Mem0형) 위에 실행 상태 계층(SKILL.state형)을 얹고, 그 사이의 경계에서 “이 사실을 참고로 보낼까, 작업대에 둘까”를 정하는 정책 하나. 이 이층 구조에서 17장이 다룰 상태 설계가 작업대 쪽의 설계도가 된다. 12장에서는 이 계보의 다른 가지 — 창을 늘리고 압축하는 흐름 — 를 따라가며, 왜 그 가지가 9장의 참사로 이어졌는지를 본다.

DAY

12

제12장 · 긴 문맥의 착각과 압축의 함정

“원문 연구: Lost in the Middle(Liu 외, TACL 2024) · ∞Bench(Zhang 외, ACL 2024) · StreamingLLM(Xiao 외, ICLR 2024) · LLMingua(Jiang 외, EMNLP 2023) · Selective Context(Li 외, EMNLP 2023) — SKILL.state §2.4 참고문헌의 계보”

한 줄 요약

창을 늘리는 흐름과 압축하는 흐름 — SKILL.state의 참고문헌이 가리키는 두 이웃을 읽으면, 이 논문이 왜 세 번째 길을 골랐는지가 보인다.

착각 — 창이 넓다고 다 보이지 않는다

“문맥 창이 백만 토큰인데요?” — 이 장의 출발점이 되는 반문이다. Lost in the Middle은 이 반문에 대한 가장 유명한 반박이다. 정답 문서를 긴 입력의 앞, 중간, 뒤에 배치해 가며 성능을 재자 성능이 U자를 그렸다 — 앞과 뒤에서는 잘 읽던 모델이 한가운데서는 못 읽는다. 위치가 능력을 바꾸는 이 현상은 긴 문맥을 “그냥 들이붓는” 전략의 계산서다.

더 신랄한 발견이 뒤따른다. 상관없는 문서를 더 넣으면 성능이 내려간다는 것 — 정보가 늘어 총량이 커질수록 모델은 못 읽는다는 방향의 사실이다. ∞Bench는 이 어두움을 극단까지 끌고 간다 — 평균 10만 토큰을 넘는 과제들에서 긴 창을 자랑하던 모델들조두 점수가 무너졌다. 창이 폭과 읽는 능력은 다른 축이다.

이 두 연구가 SKILL.state의 서두와 만나는 지점을 보라. 이력이 자라는 실행은 결국 “관련 문서가 어디쯤 있는지 모르는 초장문 입력”을 상시적으로 만든다. 7장의 노이즈 실험에서 ReAct형이 0.76에서 0.11로 떨어진 곡선은 이 문헌이 예언한 부식의 실행 버전이다.

연명 — 스트리밍의 절묘한 타협

StreamingLLM은 다른 질문을 던진다 — 아주 긴 입력을 어차피 다 못 읽는다면, 읽을 수 있는 만큼만 들고 가는 설계는 없는가. 그 답이 어텐션 싱크다 — 모델의 주의가 처음 몇 토큰에 닿을 내리는 성질을 이용해, 그 닿은 영구히 붙잡고 나머지는 최근 창만 활처럼 당겨 쓰는 것. 이 설계로 사백만 토큰이 넘는 입력을 흘려 보내도 추론이 안정적으로 버틴다.

대가도 분명하다. 창 밖으로 나간 것은 회상되지 않는다. 스트리밍은 **과거를 잊어도 되는 실행**의 기술이지 과거를 필요로 하는 실행의 기술이 아니다. 9장의 슬라이딩 윈도우가 0.18로 추락한 자리가 정확히 여기다 — 초기 재고 배치가 창밖으로 나가는 순간 창고 과제는 풀 수 없어진다. 절단이 무엇을 자를지 모르는 무딘 칼이라는 것.

함정 — 통계는 구조를 못 지킨다

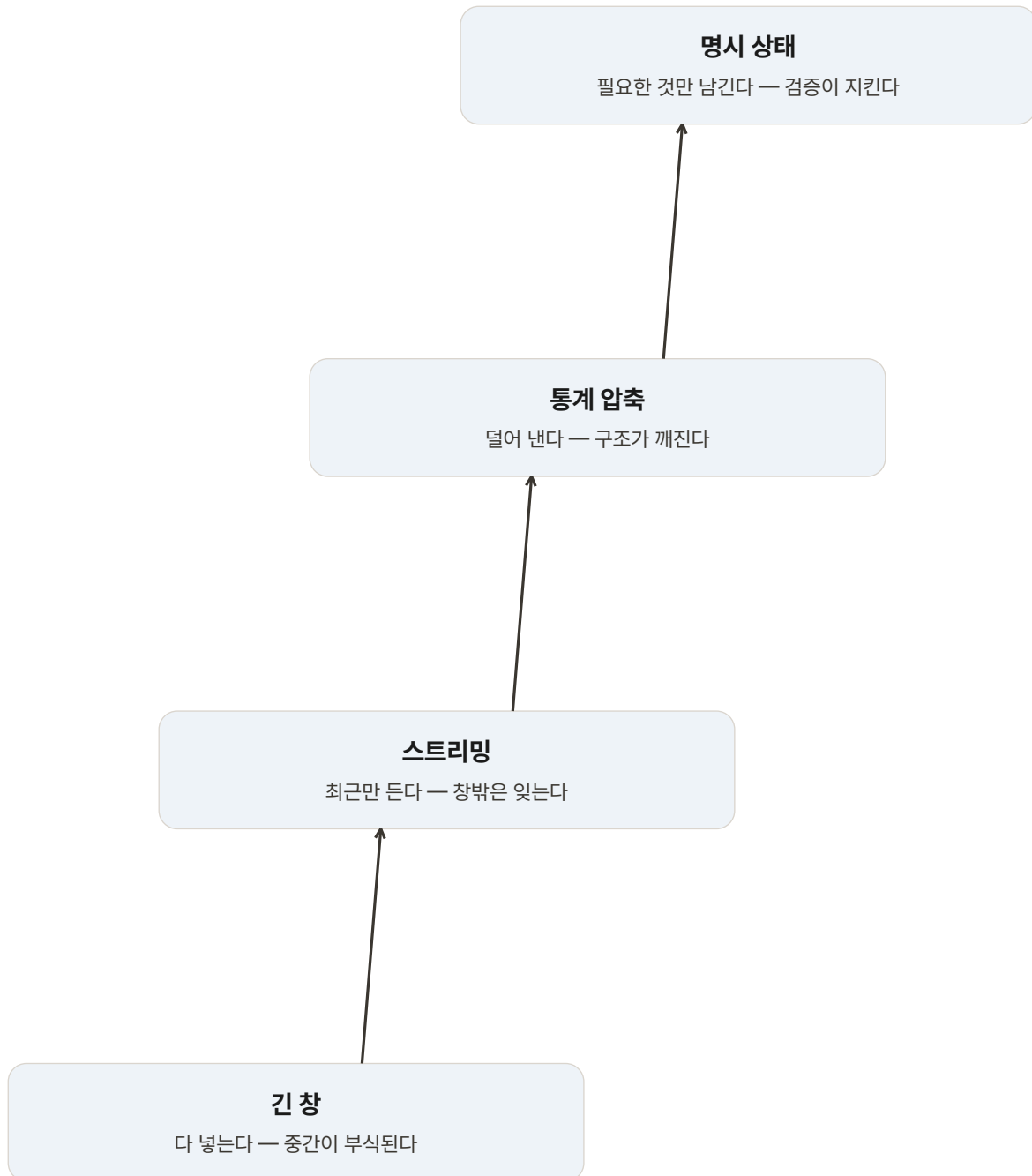
압축 계열의 접근은 우아하다. LLMingua는 작은 언어 모델의 혼란도를 자로 삼아 — 예측하기 쉬운 토큰은 복제 가능한 토큰이라 여겨 — 프롬프트에서 토큰을 가지치기한다. 거친 데서 고운 데로, 문단 단위부터 토큰 단위까지 좁혀 가는 다단계 계획 아래 최대 스무 배 압축을 주장했다. Selective Context는 같은 정신을 자기정보량으로 구현한다.

그런데 9장의 예산 매칭 대조에서 이 계열의 성적은 0.22였다. 혼란도가 낮은 토큰 — 통계적으로 심심한 토큰 — 이 실행에는 결정적일 수 있다는 것이 무너진 지점이다. “shelf_42”는 언어 모델에게 지루한 문자열이지만 창고 세계에서는 정답 그 자체다. 압축의 자는 언어의 자이지 세계의 자가 아니다. 이 함정을 논문 표현으로 옮기면 — 통계적 압축기가 파괴하는 정확한 관계 의존성을 구조화된 상태 유지는 보존한다.

FOUR ANSWERS

긴 실행의 네 가지 처방은 다른 층에 닿는다

창을 늘리고, 흘러 보내고, 잘라 내고, 상태로 남기는 일은 경쟁 대안이 아니라 내림차순으로 근본적인 치료다.



편집 요약: 이 장의 네 흐름 대비를 재배열한 도식이다.

나의 해석 — 세 흐름은 같은 진단의 세 처방전

이 장의 문헌들을 한 줄로 꿰면 이렇다 — 셋 다 “긴 문맥이 병”이라는 같은 진단서를 받고 처방이 다르다. 창 확장은 병을 무시하고 체력을 기르는 처방, 스트리밍은 아픈 기억과 함께 살기를 택하는 처방, 압축은 병소를 통계로 도려내는 처방. SKILL.state는 수술이다 — 필요한 조직만 남기고 나머지를 떼어 낸다. 수술이 늘 옳은 것은 아니다. 그러나 9장의 대조가 보여주듯 같은 예산에서 절망적 삼총사와 0.94가 같았다면, 병의 자리를 정확히 짚은 쪽은 수술이다.

실무에 돌아와서 — 이 네 처방은 실제로 섞어 쓰는 게 정답인 경우가 많다. 아주 긴 정적 참조(코드베이스 문서, 제품 매뉴얼)는 검색과 창 확장이 맞고, 실시간 로그 스트림은 스트리밍이 맞는다. 문제는 이런 자료들을 **실행의 기억**에도 그대로 쓰는 관습이었다. 다음 장에서는 이 병의 가장 오래된 선례 — 대화 상태 추적 — 를 찾아가, 상태를 믿는 설계가 에이전트 이전에 이미 한 세대를 굴렀다는 것을 본다.

DAY

13

제13장 · 대화 상태 추적에서 온 유전자

“원문 연구: DSTC(Williams 외, SIGDIAL 2013) · DSTC-2(Henderson 외, 2014) · TRADE(Wu 외, ACL 2019) · Soloist(Hosseini-Asl 외, NeurIPS 2020) · SGD(Rastogi 외, AAAI 2020) · TripPy(Heck 외, SIGDIAL 2020) — SKILL.state §2.3 참고문헌의 계보”

한 줄 요약

상태를 믿는 실행은 새것이 아니다 — 예약과 주문을 다루던 대화 상태 추적은 반세기 가까운 그 설계를 굴러 왔고, SKILL.state는 그 유전자를 자율 에이전트로 옮겨 심었다.

대화 상태 추적이란 무엇인가

에이전트 이전에, 언어 모델 이전에도 — 대화 시스템은 상태를 다루는 기술이었다. 대화 상태 추적(DST)은 작업 지향 대화에서 “지금까지 확정된 사실”을 슬롯-값의 장부로 유지하는 기술이다. 고객이 “내일로 바꿔주세요”라 말하면 날짜 슬롯이 갱신되고, “아니다, 모레로 하죠”라 말하면 다시 갱신된다. 발화의 홍수 속에서 요구사항의 현재값을 쫓는 일 — DSTC 시리즈가 이 기술의 공개 경연장이었다.

이 계보의 후반부가 정교해지는 모양새가 흥미롭다. TRADE는 도메인을 넘어 상태를 통째로 생성하는 복사 기반 생성기를 세웠고, SGD는 스키마로 짜인 다도메인 세계를 표준 과제로 만들었으며, TripPy는 세 곳에서 값을 복사하는 전략 — 맥락에서, 데이터베이스에서, 그리고 **직전 상태에서** — 로 갱신의 문법을 다듬었다. 특히 마지막 복사원을 눈여겨보라. 대부분의 턴에서 대부분의 슬롯은 그대로인데 매번 다시 쓰는 것은 낭비다 — 바뀐 것만 고치라는 3장의 병합 연산자와 정신이 같다.

상태 추적은 매 턴 장부만 고친다

대화 상태 추적(DST)의 갱신 순환 — 발화가 와도 바뀌는 것은 슬롯뿐이다

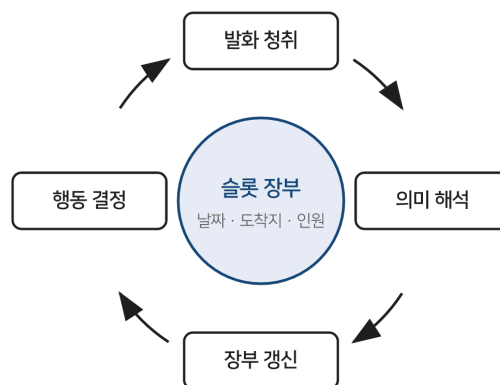


그림 13-1 상태 추적의 순환 — 매 턴 장부만 고친다

답음 — 그리고 세 가지 근본 차이

SKILL.state 논문이 이 계보를 두고 내린 구분이 정확해서 그대로 옮긴다. 답은 것 — 둘 다 구조화된 상태를 유지한다는 것. 다른 것은 셋이다.

상태의 지위가 다르다. DST에서 상태는 전사본 곁에 놓인 보조 장부다. 시스템은 상태와 함께 대화 전체를 품고 있으며, 상태는 채점과 라우팅을 돕는 요약본에 가깝다. SKILL.state에서 상태는 유일한 기질이다 — 전사본은 존재하지 않고, 상태가 곧 다음 행동의 전부를 결정한다. 보조 자료와 유일한 재판판의 차이이다.

세계가 다르다. DST의 세계는 준정적이다 — 슬롯의 뜻이 바뀌지 않고, 도메인이 제자리에 있으며, 갱신이 턴 단위로 정렬된다. SKILL.state의 세계는 동적이다 — 관찰이 마구 도착하고, 세계가 행위자 밖에서 바뀌며(7장), 스키마조차 도메인마다 다시 짠다. 조용한 사무실과 폭풍 치는 작업장의 차이이다.

갱신의 책임이 다르다. DST의 갱신은 모델의 산출 그 자체로 확정된다. SKILL.state의 갱신은 결정론적 런타임의 검증을 통과해야 한다(3장). 장부가 틀리면 틀린 대로 다음 턴이 태어나는 쪽과, 틀린 장부가 아예 등재되지 않는 쪽 — 15장의 오류 분석이 이 차이의 값을 매긴다.

나의 해석 — 충분통계량의 두 세대

2장에서 심어 둔 읽기를 여기서 회수한다. 나는 DST와 SKILL.state를 같은 개념의 두 세대로 본다 — **충분통계량의 세대**. DST 세대는 통계학의 이 오래된 개념을 사람의 발화에 적용해 예약과 주문을 다스렸다. SKILL.state 세대는 같은 개념을 도구와 환경에 적용해 실행 전체를 다스린다. 세대 차이의 핵심은 대상이 아니라 각오다 — 이전 세대는 상태가 충분하다고 믿되 전사본을 안전벨트로 품었고, 새 세대는 안전벨트를 벗었다. 벗은 대가로 4장의 선형 비용과 7장의 즉시 회복을 얻었다.

이 독법이 주는 실무의 교훈 하나 — 이미 DST를 하고 있던 시스템은 많다. 챗봇의 세션 변수, CRM의 고객 상태, 워크플로 엔진의 변수 테이블이 다 그것이다. 그 시스템들이 아직도 매 턴 전체 대화를 문맥에 실어 나른다면, 그것이 전환의 출발점이다. 상태는 이미 있는데 그릇만 바꾸면 되는 수가 많다. 17장에서 이 전환의 절차를 짚는다.

DAY

14

제14장 · 시험대의 설계

“원문 연구: InterCode(Yang 외, NeurIPS 2023) · τ -Bench(Yao 외, arXiv:2406.12045) — SKILL.state §4.2
가 세운 시험대의 설계도를 파헤친다”

한 줄 요약

8장의 결과가 나온 두 시험대는 우연히 뽑힌 게 아니다 — 하나는 반복 억제의 이득을 극대화하는 시행착오 세계, 하나는 상태 정합성의 이득을 극대화하는 규칙 세계다.

왜 벤치마크를 다시 읽는가

벤치마크는 흔히 결과의 배경으로만 읽힌다. 그러나 좋은 벤치마크는 주장의 일부다 — 무엇을 시험하는지가 무엇이 입증되는지를 결정하니까. 8장에서 SKILL.state가 세 경기에서 이겼다고 할 때, 그 세 경기가 어떤 운동장인지를 몰라 하면 승리의 성격을 오독하게 된다. 이 장은 5장의 통제 세계와 짝을 이루는 야전 두 개를 설계도 수준에서 뜯어 본다. 인용의 정확성을 위해 밝혀두면 이 장의 서술은 두 벤치마크 원문문에 대한 내 요약이며, 결과 숫자는 8장에서 이미 다루었다.

InterCode — 시행착오의 표준화

InterCode의 출발 문장이 사랑스럽다 — 코드를 **행동**으로, 실행 피드백을 **관찰**로. 코딩을 닫힌 문제(문제를 주고 정답을 받는)가 아니라 열린 상호작용(명령을 치고 결과를 보고 다시 치는)으로 재정의한 것이다. 에이전트는 격리된 도커 컨테이너 안에서 명령을 실행하고, 환경은 실행 출력과 상태를 돌려준다. 데이터베이스 질의 트랙과 리눅스 bash 트랙이 있고, 그 bash 트랙의 한가운데에 CTF 하위 과제 — 100개의 숨겨진 플래그 회수 — 가 앉아 있다.

이 설계가 상태 관리를 시험하는 데 얼마나 좋은지 보려면, CTF의 실패 모드를 상상해 보면 된다. 같은 도구를 같은 순서로 다시 돌리는 것(기억 없음), 한 번 깨진 가설을 다시 굴리는 것(망각 없음), 발견한 단서를 잃는 것(기록 없음). 세 실패 모두 이력이 넘치거나 상태가 부실할 때 발생한다. InterCode는 특별히 상태를 겨냥해 만들어진 벤치마크가 아닌데도 — 상호작용을 표준화했다는 사실 자체가 — 상태 관리의 검증장이 된다. 좋은 시험대의 무게가 이렇다.

τ -Bench — 규칙 아래의 트랜잭션

τ -Bench가 노리는 취약점은 다르다. 도구를 쓰는 것과 **쓸 만한 상태를 유지하는 것**은 다른 일임을 보여주는 데 이 벤치마크의 사명이 있다. 구성이 삼각형이다 — 에이전트, 도구(관계형 데이터베이스 조회와 트랜잭션), 그리고 시뮬레이션된 사용자. 소매와 항공 두 도메인의 실무 시나리오에서 사용자의 말은 바뀌고, 데이터베이스의 응답은 크고, 비즈니스 정책은 뻑뻑하다. 환불 규정을 어기며 친절해진 답은 성공이 아니고, 정책을 지키며 고객 의도를 데이터베이스의 최종 상태로 옮겨 놓은 실행만 성공이다. 채점이 프로그램으로 되어 있다는 것이 두 번째 미덕 — 최종 상태의 정합성과 정책 준수를 기계적으로 판정하니 채점의 주관이 없다.

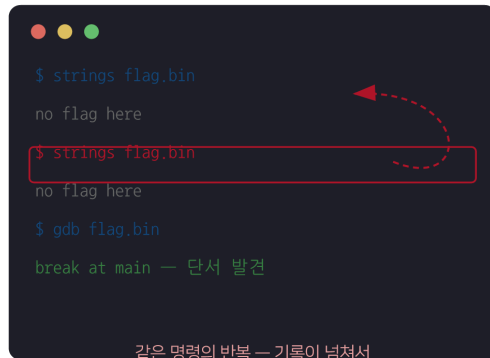
그리고 이 벤치마크가 남긴 가장 유명한 발견을 짚고 지나가야 한다 — **일관성의 벽**. 한 번 잘하는 것과 매번 잘하는 것은 다른 축이라는 것. 반복 실행 전체의 성공(pass^k로 표기하는, k번 연속 성공의 확률)은 k가 늘수록 처참하게 떨어지는데, 강한 모델조차 예외가 아니었다. 데모 한 번의 영웅과 운용 첫날의 평균 사이에 이 격차가 끼어 있다. 상태 기반 실행이 이 벤치마크에서 강한 까닭도 여기에 닿아 있다 — 결과의 분산이 이력 재생의 운이 아니라 상태의 정확성에서 오는 구조는, 운에 기대는 구조보다 반복에 강하다.

구분	InterCode CTF	τ -Bench
세계의 모양	격리된 도커 컨테이너의 bash 세계	사용자·도구·데이터베이스의 삼각형
실패의 양상	같은 명령 반복·깨진 가설 재시도·단서 망각	정책 위반·DB 불일치·사용자 의도 누락
채점	플래그의 정확한 일치 (이진)	최종 상태·정책 준수의 기계 판정
상태 관리가 점수가 되는 길	반복 억제 → 시도의 다양화	정합성 유지 → 무결한 트랜잭션

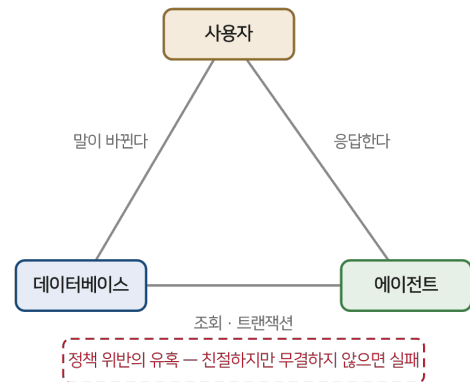
구분	InterCode CTF	τ -Bench
저자들이 뽑은 이유	시행착오 세계에서 실행 역학 검증	오염된 문맥 속 트랜잭션 검증

두 시험대는 서로 다른 실패를 유혹한다

InterCode CTF는 반복의 유혹으로, τ -Bench는 위반의 유혹으로 상태 관리를 시험한다



InterCode CTF — 시행착오의 터미널



Sierra τ -Bench — 정책 아래의 트랜잭션

그림 14-1 두 시험대 — 반복의 유혹과 위반의 유혹

나의 해석 — 시험대는 주장의 문법이다

이 장을 다 읽고 8장의 표로 돌아가면 보이는 것이 있다. SKILL.state의 승리가 세 경기에서 나왔다는 말은, 사실 **두 종류의 승리**였다는 말이다. CTF의 승리는 반복을 줄여 시도의 다양성을 늘린 이야기(탐색의 개선)고, τ -Bench의 승리는 오염된 문맥 속에서 정합성을 지킨 이야기(신뢰성의 개선)다. 하나의 설계가 성질이 다른 두 시험대에서 같은 방향의 결과를 냈다는 것 — 이중 증언이 논문의 설득력의 뼈대다.

실무로 가져갈 교훈도 문법의 교훈이다. 당신의 에이전트를 시험할 때 시험대의 성격을 먼저 쓰라 — 반복의 유혹이 있는가, 위반의 유혹이 있는가. 그리고 개선을 주장할 때는 이 장의 표처럼 어떤 유혹을 이겼는지로 문장을 쓰라. “정확도가 올랐다”는 문장은 약하고, “같은 파일을 다시 읽는 빈도가 줄어 탐색이 다양해졌다”는 문장은 강하다. 벤치마크를 설계도로 읽는 습관이 주장의 문장을 바꾼다.

DAY

15

제15장 · 작은 모델은 어디서 무너지나

“원문: SKILL.state — arXiv:2608.26263 · §5.7, 표 7·8”

한 줄 요약

소형 모델에서 구조의 이득은 더 크지만 천장은 낮다 — 실패 로그의 분포는 문제가 추론 능력이 아니라 구조화 출력 준수 임을 가리킨다.

두 개의 작은 모델, 같은 문양

지금까지의 실험은 Gemini-3-Flash였다. 이 논문의 주장이 진짜라면 모델이 바뀌어도 방향은 유지되어야 한다. 표 7과 표 8이 그 확인이다 — 개방 가중치의 Gemma-4-31B-it와 Qwen-3-8B-it를 참고 환경에 세운 결과다.

모델	지평	프롬프트	메모리	스테이트풀	SKILL.state
Gemma-4-31B	10	0.90	0.85	0.90	0.98
	25	0.64	0.72	0.76	0.84
	50	0.31	0.41	0.55	0.68
	100	0.21	0.24	0.42	0.42
Qwen-3-8B	10	0.84	0.80	0.84	0.94
	25	0.54	0.62	0.66	0.76
	50	0.24	0.33	0.44	0.58
	100	0.15	0.18	0.31	0.34

소형 모델에서도 방향은 같다 — 폭은 크게

참고 환경 점수 · 논문 표 7(Gemma-4-31B)·표 8(Qwen-3-8B) · 가로축은 실행 지평 T

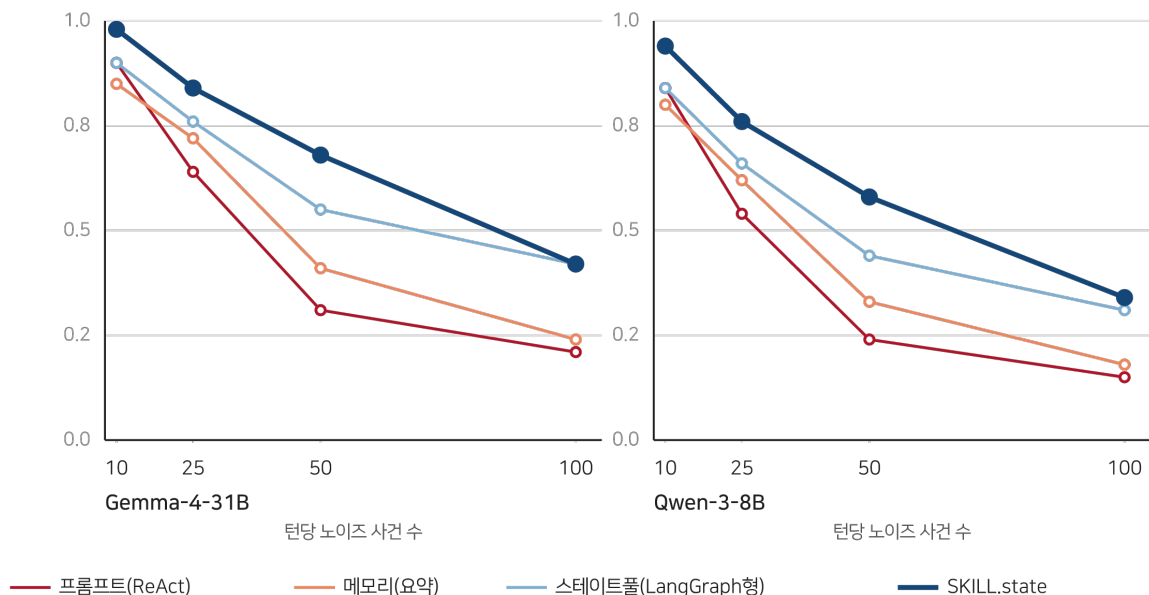


그림 15-1 소형 모델의 스케일링 — 같은 방향, 낮은 천장

읽을 거리가 둘 있다. 하나 — 모든 지평에서 SKILL.state가 1위다. 작은 모델일수록 격차가 벌어진다는 것은 이 구조가 모델 능력을 보완하는 쪽이라는 뜻이다. 지평 50의 Gemma에서 베이스라인 최강인 스테이트풀이 0.55인데 SKILL.state는 0.68 — 31B 모델이 구조의 도움으로 앞선 런타임을 이긴다.

둘 — 지평 100의 Gemma에서 스테이트풀과 SKILL.state가 0.42로 동률이다. Qwen의 0.31 대 0.34도 거의 붙어 있다. 천장의 존재다. 구조가 아무리 좋아도 상태 갱신 자체를 틀리는 모델에게는 그 이상이 안 된다. 그리고 이 천장의 원인을 논문은 실패 로그를 뜯어 찾았다.

오류의 세 얼굴

Gemma-4-31B의 지평 100 실패 로그를 분류한 결과가 이 논문의 가장 실용적인 발견 중 하나다.

첫째, 조기 상태 덮어쓰기·삭제 — 68%. 모델이 갱신 조각을 낼 때 기존 키를 빠뜨리는 것이 아니라 아예 누락시켜 통째로 지우는 실수. “바뀐 것만 말하기” 계약(3장)을 어기는 경우다. 가장 흔하고 가장 치명적이다 — 장부의 페이지가 사라지면 이후의 모든 조치가 틀린다.

둘째, 스키마 이해·형식 강제 — 20%. 중첩 리스트와 딕셔너리의 구분을 흐리는 경우. 상태의 모양 자체를 잘못 이해하는 실패다.

셋째, JSON 구문 사고 — 12%. 심표 하나, 괄호 하나의 실수. 문법 파탄은 검증 게이트에서 걸리니 영구 상태는 오염되지 않지만, 재시도에 토큰이 소모된다.

소형 모델의 실패는 준수의 문제다

Gemma-4-31B · 지평 100 실패 로그의 분포 — 논문 §5.7

갱신 계약 위반이 10건 중 9건 — 추론 능력이 아니라 형식 준수가 무너진다

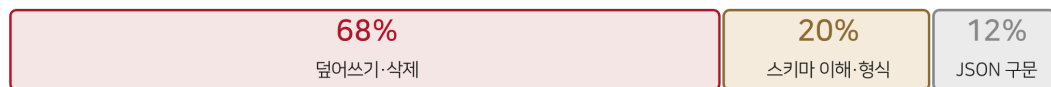


그림 15-2 오류 분류 — 10건 중 9건이 갱신 계약 위반

진단 — 추론이 아니라 준수다

이 분포의 뜻은 명확하다. 소형 모델의 한계는 **생각이 부족해서가 아니라 계약을 지키는 일이 부족해서**다. 셋 다 구조화된 출력의 형식적 준수에 속하는 실패라는 것 — 그래서 논문의 처방은 모델을 키우는 것이 아니라 **문법 제약 디코딩**이다. 가능한 출력의 공간을 문법 자체로 좁혀 구문 오류를 원천 봉쇄하고, 모델에게 의미적 전이에만 전념하게 하는 기술. 68%의 덮어쓰기까지는 못 미치지만 12%의 구문 사고를 0으로 만들고, 병합 연산자의 설계를 조이면 최빈 오류의 폭도 좁힐 수 있다.

이 진단은 3장의 분업을 다시 떠올리게 한다 — 상태의 무결성을 런타임이 지키는 대가로, 모델의 실수가 시스템의 실수로 번지는 통로가 닫혀 있었던 것. 소형 모델의 천장은 그 통로가 닫혀 있어도 여전히 존재하지만, 그 천장의 재질이 “추론력”이 아니라 “준수력”이라는 것이 이 장의 결론이다.

나의 해석 — 비대칭의 뜻

표의 비대칭이 많이 많다. 짧은 지평에서의 격차는 크고 긴 지평에서는 좁아진다 — 지평 100의 Gemma에서 1위와 2위가 동률이 되는 것. 내 독해는 이렇다. 구조의 이득은 두 몫이다 — 이력 병리를 없애는 몫(모든 지평에서 유효)과 상태 갱신을 정확히 하는 몫(모델 준수력에 비례). 대형 모델은 두 몫을 다 받는데, 소형 모델은 첫 몫은 받고 둘째 몫의 한계에 부딪힌다. 그래서 “방향은 같고 천장은 낮다”가 되는 것.

이 독해의 실무적 딸림표 — 이 구조를 소형 모델로 돌리려면 준수력부터 올려야 한다. 문법 제약 디코딩, 스키마 검증의 재 시도 예산, 더 단순한 스키마. 모델을 바꾸는 것이 마지막 수단이지 첫 수단이 아니다. 17장의 체크리스트에 이 순서가 반영되어 있다.

DAY

16

제16장 · 이 설계가 무너지는 세 지점

“원문: SKILL.state — arXiv:2608.26263 · §7 Limitations”

한 줄 요약

“상태가 충분통계량이다”라는 전제가 깨지는 곳에서 이 설계도 깨진다 — 논문 스스로 적은 세 붕괴 지점을 읽는 것이 도입을 판단하는 첫걸음이다.

전제를 먼저 분명히

SKILL.state의 모든 마법은 하나의 전제 위에 서 있다 — **과거가 미래에 영향을 주는 모든 것이 알려지는 즉시 상태로 투영될 수 있다면, 중간 추론과 대화 이력을 버리는 일은 무손실이다**. 2장에서 충분통계량이라 이름 붙인 전제다. 이 전제가 성립하는 한 폐기는 임의의 절약이 아니라 수학적 권리다.

논문의 품격이 돋보이는 대목은 이 전제가 깨지는 자리를 스스로 세 곳으로 적어 두었다는 것이다. 좋은 설계의 시그니처는 자기 적용 범위의 지도를 함께 내놓는 것이다.

첫째 — 스키마를 미리 모를 때

상태는 스키마 위에 산다. 그런데 스키마는 도메인마다 한 번 짚다고 했는데(3장), 도메인의 골격 자체가 실행 중에야 드러난다면? 탐색적 과제 — 무엇을 찾아야 하는지도 모르고 시작하는 조사, 상황이 굴러가며 형태가 정해지는 디버깅, 처음 보는 장르의 사건 분석 — 에서는 무엇을 장부에 올릴지 미리 알 수 없다. 관련 구조를 실행 중에 발견해야 하는데, 받아 쓸 스키마가 없으면 SKILL.state는 시작조차 어렵다.

이 지점에서 이 설계는 보수적이다 — 알고 있는 세계의 실행기라는 것. 15장의 실무 절차가 “스키마를 먼저 쓰라”로 시작하는 것도 이 붕괴 지점의 반영이다.

둘째 — 뒤늦은 관련성

둘째 붕괴는 더 미묘하다. 어떤 관찰은 처음 봤을 때는 아무 뜻 없는 디테일이었다가, 스무 턴 뒤에 결정적 증거가 되는 경우도 있다. SKILL.state의 문법에서 그 관찰은 이미 버려졌다 — 상태 갱신을 만들지 못한 관찰은 다음 날 갈 길로 남는데, 갈 곳이 없다. “그때 그게 중요한 줄 어떻게 알았겠냐”는 세계에서는 이력을 통째로 들고 다니는 쪽이 나을 수 있다.

다만 이 한계의 실제 무게는 재보아야 한다. 방어법이 있기 때문이다 — 확실하지 않은 관찰을 미리 상태에 올려 두는 것. 보수적인 스키마는 “나중에 중요할지 모르는 것”의 칸을 가질 수 있고, 실은 CTF의 5필드 스키마가 정확히 그런 문법이다 (tested_hypotheses는 회고를 위한 칸이 아니라 미래의 반복 억제를 위한 칸이지만, discovered_flags는 명중 여부가 뒤늦게 밝혀지는 것들의 창고다). 원시 이력 전부를 보존하는 것과 아무것도 보존하지 않는 것 사이에는, 상태에 후보를 올려 두는 중간지대가 있다.

셋째 — 궤적 자체가 대상일 때

셋째는 가장 근본적이다. 감사(auditing), 디버깅의 근거 추적, 과거 행동의 설명 — 이런 과제의 산출물은 궤적 그 자체다. 이력이 비용이 아니라 상품인 세계에서 이력을 버리는 것은 재고를 태우는 것이다. SKILL.state의 관점에서 보면 이런 과제는 “실행”이 아니라 “기록의 판독”이므로 설계의 바깥이다 — 다만 바깥이라고 선을 긋는 순간, 그 바깥의 일을 하는 시스템에 이 설계를 이식하면 안 된다는 경계도 함께 생긴다. 실행 계층과 감사 계층을 분리하라는 교훈으로 읽는 것이 옳다. 실행은 상태로 굴리고, 감사가 필요하면 상태 전이의 로그(이력이 아니라 갱신의 연쇄)를 별도로 쌓는다 — 전사본과는 다른, 훨씬 얇은 로그다.

그리고 두 개의 미완

논문은 여기에 미완을 두 개 더 적는다. **다중 에이전트** — 공유 상태가 조율의 중심 기질이 되는 확장은 자연스러워 보이지만, 동시 쓰기가 생기면 병합 연산자는 결정론적 충돌 해결 규칙을 가져야 하는데 지금은 외통수뜨리 검증만 있다. **제약 디코딩** — 15장의 처방으로, 구문 오류를 문법으로 봉쇄하는 일이 후속 과제로 남아 있다.

전제가 깨지는 세 지점 — 충분통계량의 우산

상태가 미래에 필요한 전부를 품는 한 폐기는 무손실이다 · 논문 §7

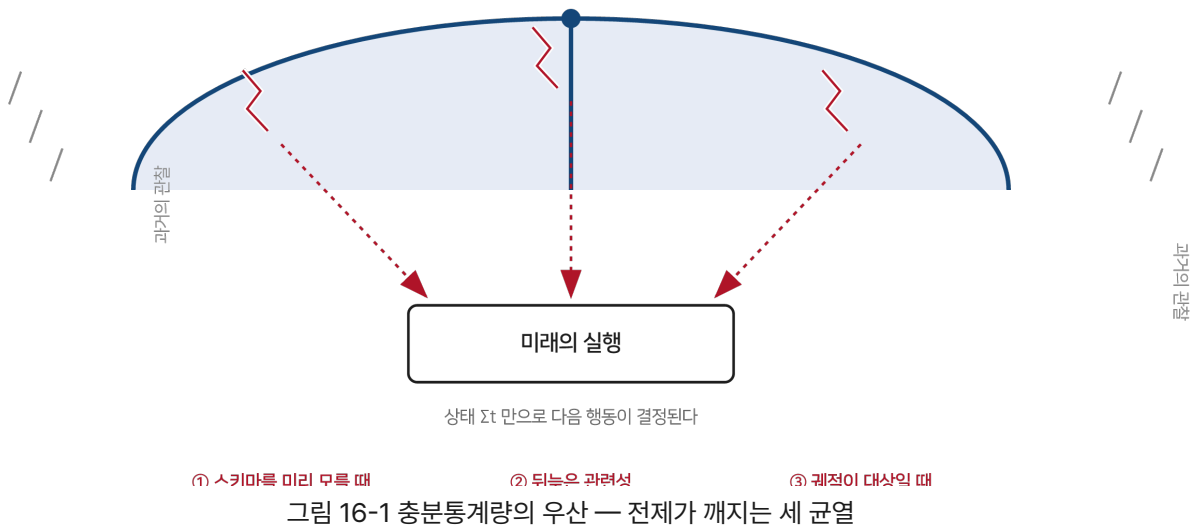


그림 16-1 충분통계량의 우산 — 전제가 깨지는 세 균열

나의 해석 — 붕괴 지점은 도입 질문이 아니라 설계 질문

이 장의 세 지점을 “그래서 못 믿는다”의 근거로 쓰는 것은 잘못 읽은 것이다. 올바른 용법은 이것을 **입구의 분류기**로 쓰는 것이다 — 내 과제가 세 지점 어디에도 걸리지 않는가를 먼저 묻는 것. 걸리지 않는다면 이 설계의 이득을 통째로 받을 수 있다. 걸린다면 세 지점 각각에 대응 설계가 있다 — 스키마 발견이 필요하면 이질적 이력을 한동안 허용하는 과도기, 뒤늦은 관련성이 걱정되면 후보 칸을 둔 보수적 스키마, 궤적이 대상이면 상태 전이 로그라는 얇은 별도 계층.

그리고 네 장의 계보 읽기(10~13장)를 여기에 얹으면 이 한계 장이 완성된다 — 세 붕괴 지점은 각각 계보의 다른 가지가 담당하던 땅이다. 스키마 발견은 ReAct의 탐색이, 뒤늦은 관련성은 장기 기억 계열(Mem0·MemoryBank)이, 궤적 보존은 감사 로그가 맡으면 된다. SKILL.state는 만능 대체재가 아니라 분업의 한 자리다 — 그 자리의 경계를 이토록 정직하게 그려 둔 것이 이 논문을 신뢰하는 이유다.

DAY

17

제17장 · 상태 설계 역설

“원문: SKILL.state — arXiv:2608.26263 전편의 실무 적용 (이 장은 논문의 결과를 딴고 쓴 저자의 가이드다)”

한 줄 요약

이 책의 실무 요약 — 무엇을 상태에 올릴지 정하는 질문 여섯 개와, 기존 대화형 에이전트를 상태형으로 옮기는 절차.

상태 설계의 질문 여섯

16장의 분류기를 통과했다면 — 과제가 절차적이고, 스키마를 미리 그릴 수 있고, 궤적 자체가 산출물이 아니라면 — 남은 일은 상태를 짜는 것이다. 논문과 계보에서 뽑은 여섯 개의 질문이 전부다.

하나 — 미래가 필요로 하는가. 상태에 올릴 자격은 딱 이것뿐이다. 다음 행동을 고르는 데 참조되지 않는 사실은 아무리 흥미로워도 상태가 아니라 잡동사니다. “이 정보가 뒤에 또 쓰이는가”를 모든 필드에 던져 보라 — 걸리는 필드는 지우는 것이 상태의 다이어트다.

둘 — 도메인 전체를 덮는가. 일 건건이 스키마를 새로 짜지 마라. 창고의 500선반이 그랬고 CTF의 100과제가 그랬듯, 도메인의 뼈대 하나면 수많은 일을 받친다. 스키마가 일 따라 늘어난다면 도메인 모델링이 아니라 코딩을 하고 있는 것이다.

셋 — 갱신이 문장인가 명령인가. 상태 갱신은 산문이 아니라 연산이어야 한다. “3번 선반쯤에 있었던 것 같다”가 아니라 `"shelf_3": null`이다. 요약 상한이 0.52에서 멈춘 이유(9장)를 문법의 차이로 다시 읽는 것이다.

넷 — 지워야 할 것도 말할 수 있는가. null-삭제 문법을 가져라. 사실은 생기기만 하지 않는다 — 출하되고, 취소되고, 뒤집힌다. 갱신 계약이 삭제 동사를 모르면 상태는 쓰레기장이 된다.

다섯 — 틀린 갱신이 어디서 멈추는가. 검증의 자리를 정해 두어라. 스키마 위반·자료형 위반은 런타임이, 의미의 오류는 다음 관찰이 잡는다. 모델의 실수가 영구 장부로 통과하는 경로가 하나라도 있으면 이 설계의 보험(3장의 분업)은 끊긴다.

여섯 — 회복은 즉시인가. 세계가 내 동작 밖에서 바뀌는 통로를 만들어 두고, 그 통로로 온 소식이 상태를 즉시 갱신하는지 시험해라. 7장의 실험 3이 재현 규격이다 — 회복 단계 0이 목표이고, 몇 턴을 헤매는지가 설계의 성적표다.

이행 절차 — 다섯 걸음

이미 돌아가는 대화형 에이전트를 상태형으로 옮기는 절차. 논문의 구조를 그대로 실무의 단계로 옮긴 것이다.

걸음 1 · 반복을 센다. 로그에서 같은 행동의 반복 — 다시 읽은 파일, 다시 날린 질의, 다시 실패한 시도 — 을 센다(8장). 이 숫자가 이행의 정당화이자 이행 뒤 성과의 기준선이 된다.

걸음 2 · 스키마를 쓴다. 도메인에서 미래의 실행을 좌우하는 것들만 뽑아 필드로 만든다. 다섯 개 안팎에서 시작하라 — CTF가 그랬다. 후보 칸(중요할지 모르는 것)을 하나 둘 수 있으나 전부 후보면 스키마가 아니다.

걸음 3 · 갱신 계약을 문다. 매 턴의 산출을 추론·갱신·행동의 세 묶음으로 못 박고, 갱신은 JSON 병합 조각으로만 받는다. 프롬프트에 “추론은 실행 뒤 버린다”고 미리 공지하는 것까지 논문의 문법 그대로.

걸음 4 · 검증과 병합을 런타임에 둔다. 갱신의 검증을 결정론 코드로 쓰고, null-삭제 병합을 구현한다. 재시도 예산을 정해 두고, 검증 실패가 영구 상태에 닿지 않는 것을 테스트로 못 박는다.

걸음 5 · 나란히 달려 옮긴다. 한쪽을 버리지 말고 두 런타임을 같은 과제 묶음에 돌려 정확도·턴당 입력·총 토큰을 나란히 적는다. 6장의 표가 목표 형태다. 그리고 지평을 늘려가며 격차의 방향을 본다 — 이 대조가 이행의 승인 도장이다.

이행은 반복 측정에서 지평 확장까지 여섯 걸음

측정이 정당화를, 계약과 검증이 구조를, 나란히 달리기가 승인을 만든다

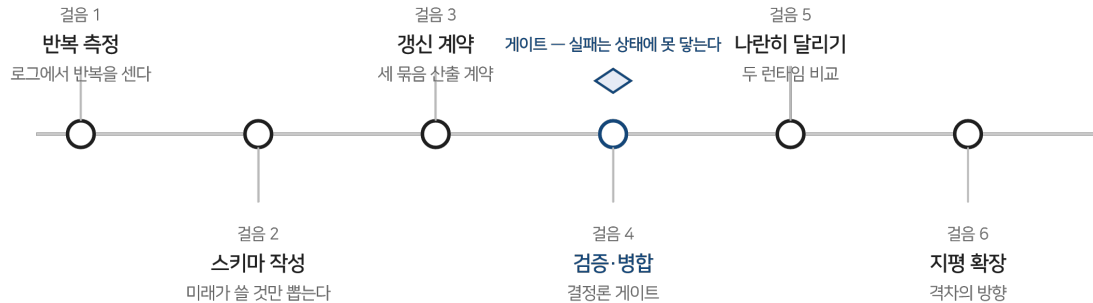


그림 17-1 이행의 여섯 걸음 — 측정에서 지평 확장까지

나의 해석 — 역설의 이름을 풀다

장의 제목을 풀 마지막 자리다. 상태 설계의 역설은 이것이다 — **가장 좋은 상태는 가장 적게 남은 상태다**. 이력을 다 갖는 것이 아니라 아무것도 안 갖는 것이 문제였으니, 해답은 “많이 기억”과 “덜 기억”의 중간이 아니라 **옳게 기억**의 다른 자리에 있다. 무엇이 남아야 하는지 아는 순간 얼마나 남는지는 자동으로 정해진다. 그래서 이 장의 절차가 상태를 “쌓는” 순서가 아니라 “고르는” 질문으로 시작하는 것이다.

마지막으로 한 점 — 이 설계는 프레임워크가 아니라 패턴이다. 오늘 밤 파이썬 백 지면으로 시작할 수 있고, 기존 시스템의 세션 변수를 재활용할 수도 있다(12장). 도입의 비용이 낮다는 것이 이 논문이 실무에게 주는 덤이다. 필요한 것은 코드가 아니라 질문이다 — “지금, 상태가 뭔가.”

DAY

18

맺음말 · 지금, 상태가 뭔가

다시 처음의 질문으로

이 책은 한 논문을 읽어 한 권을 만들었다. 출발점은 1장의 진단이었다 — 에이전트의 실행이 길어지는 순간, 실행은 추론의 문제가 아니라 시스템의 문제가 된다. 도착점은 그 시스템의 모양이었다 — 불변 명세와 현재 상태와 최신 관찰만으로 매 단계를 시작하고, 추론은 상태 갱신이 검증되는 즉시 버리는 실행기 SKILL.state.

숫자들은 그 도착을 뒷받침했다. 지평 100에서 16.2배의 토큰 격차, 지평 200에서 유지되는 0.94의 정확도, 노이즈 아래의 면적, 무음의 세계 변경 앞에서의 회복 단계 0, 같은 예산에서 압축 기법들과 갈린 0.94 대 0.18. 그리고 이 모든 숫자를 관통하는 원리는 하나였습니다 — 과거를 재생하지 않고 현재에 조건지워진 실행.

이 책이 남기고 싶은 세 가지

첫째 — **그릇은 데이터 구조다.** 대화, 요약, 압축, 상태는 모두 실행을 담는 그릇의 후보였고, 이 책은 그 후보들의 계보(10~13장)와 대조(9장)를 따라 걸었다. 어떤 시스템을 보든 “무엇이 그릇인가”부터 묻는 눈이 생겼기를 바란다. 그릇의 모양은 지수를 정하고, 지수는 청구서를 정한다.

둘째 — **버리는 것도 설계다.** 이 논문의 가장 급진적인 결정은 무엇을 넣을지가 아니라 무엇을 버릴지였다. 추론 흔적의 즉시 폐기, 지난 관찰의 미래 금지. 무엇을 남기든지의 결정은 무엇을 안 남길지의 결정 없이는 서지 않는다 — 이 원리는 에이전트의 상태만이 아니라 문서, 회의록, 지식베이스, 그리고 기억 전반에 적용되는 문법이다.

셋째 — **한계의 지도가 신뢰의 근거다.** 이 논문은 자기 전제가 깨지는 세 지점을 스스로 적었다(16장). 그 지도가 있기에 나머지 땅에서의 주장을 통째로 믿을 수 있었다. 논문을 읽는 습관에도 이 순서를 권한다 — 근거 앞에 한계를 먼저 읽는 것.

열려 있는 문

이 논문이 열어 두고 간 문도 책의 한계이자 다음 읽기의 초대장이다. 문법 제약 디코딩과 소형 모델의 준수력(15장), 다중 에이전트의 동시 쓰기와 충돌 해결(16장), 스키마를 실행 중에 발견하는 탐색적 실행(16장). 이 문들이 열리는 속도가 이 설계가 얼마나 넓은 땅의 표준이 될지를 정할 것이다.

독자에게 남기는 질문은 책 전체에서 가장 짧은 것이다. 당신의 에이전트에게 물어 보라 — 지금, 상태가 뭔가. 대답이 “지금까지의 대화”라면 이 책의 첫 장으로 돌아갈 이유가 있는 것이고, 대답이 구조화된 장부라면 17장의 여섯 질문으로 그 장부를 다시 검열할 차례다. 그 질문 하나가 이 책 전체를 대신한다.